



Guía de referencia SQL de SAP HANA Cloud, base de datos de SAP HANA

Generado el: 2026-06-06 23:22:43 GMT+0000

SAP HANA Cloud, SAP HANA Database | QRC 1/2026

Público

Contenido original: https://help.sap.com/docs/HANA_CLOUD_DATABASE/c1d3f60099654ecfb3fe36ac93c121bb?locale=en-US&state=PRODUCTION&version=2026_1_QRC

Exención de responsabilidades sobre la traducción automática

Para su comodidad, este documento PDF se ha traducido de forma automática. SAP no ofrece ninguna garantía respecto a la precisión o integridad de la traducción automática. La versión original en inglés tiene prioridad.

Si desea hacer algún comentario al respecto, utilice la opción de feedback del SAP Help Portal disponible en la versión HTML de este documento.

Advertencia

Este documento lo ha generado el SAP Help Portal y es una versión incompleta de la documentación oficial del producto de SAP. La información incluida en la documentación personalizada puede no reflejar la disposición de los temas en SAP Help Portal y puede carecer de aspectos y/o correlaciones importantes con otros temas. Por este motivo, no se recomienda para uso productivo.

Para obtener más información, visite <https://help.sap.com/docs/disclaimer>.

Funciones SQL

Documenta las funciones SQL integradas que se proporcionan con SAP HANA.

Las funciones analíticas se posicionan en esta guía como funciones agregadas y funciones agregadas de ventana.

[Lista alfabética de funciones](#)

[Agregar funciones](#)

Las funciones de agregación son funciones analíticas que calculan un valor agregado basado en un grupo de filas.

[Funciones de matriz](#)

Las funciones de matriz toman matrices como entrada.

[Funciones de conversión de tipo de datos](#)

Las funciones de conversión de tipo de datos convierten datos de un tipo de datos a otro.

[Funciones de fecha y hora](#)

Las funciones de fecha y hora realizan operaciones en tipos de datos de fecha y hora o información de fecha u hora de devolución.

[Funciones de jerarquía](#)

Las funciones de jerarquía le permiten trabajar con datos jerárquicos, como tablas con filas dispuestas en un árbol o gráfico dirigido.

[Funciones JSON](#)

Las funciones JSON son funciones que devuelven u operan en datos JSON.

[Otras funciones](#)

SAP HANA admite muchas funciones que devuelven valores del sistema y realizan varias operaciones en valores, expresiones y valores de retorno de otras funciones.

[Funciones numéricas](#)

Las funciones numéricas realizan operaciones matemáticas en tipos de datos numéricos o información numérica de retorno.

[Funciones de seguridad](#)

Las funciones de seguridad proporcionan una funcionalidad especial para fines de seguridad.

[Funciones de datos de serie](#)

Las funciones de datos de serie proporcionan una funcionalidad especial para los datos de serie y las tablas de serie.

[Funciones de cadena](#)

Las funciones de cadena realizan la extracción y manipulación de cadenas, o devuelven información sobre cadenas.

[Funciones vectoriales](#)

Las funciones vectoriales realizan cálculos en vectores.

[Funciones de ventana y la especificación de ventana](#)

Las funciones de ventana le permiten realizar operaciones analíticas en un conjunto de filas de entrada.

SUBSTR_AFTER Función (cadena)

Devuelve una subcadena de una cadena especificada que sigue la primera ocurrencia del patrón especificado.

Sintaxis

`SUBSTR_AFTER(<string>, <pattern>)`

Elementos de sintaxis

<string>

Especifica la cadena que contiene la subcadena.

<pattern>

Especifica la ocurrencia dentro de la cadena que marca el lugar inmediatamente antes de la subcadena devuelta.

Descripción

Devuelve una subcadena de la cadena especificada que sigue la primera ocurrencia del patrón especificado.

- Si **<string>** no contiene la subcadena **<pattern>**, se devuelve una cadena vacía.
- Si **<pattern>** es una cadena vacía, se devuelve **<string>**.
- Si **<string>** o **<pattern>** es NULL, entonces se devuelve NULL.

Ejemplo

Este ejemplo devuelve Amigo, la parte de la cadena dada que está a la derecha de la primera ocurrencia de Mi:

```
SELECT SUBSTR_AFTER ('Hello My Friend','My ') "substr after" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función SUBSTR_BEFORE (cadena)

Devuelve una subcadena de una cadena especificada antes de la primera ocurrencia del patrón especificado.

Sintaxis

SUBSTR_BEFORE(**<string>**, **<pattern>**)

Elementos de sintaxis

<string>

Especifica la cadena que contiene la subcadena.

<pattern>

Especifica la ocurrencia dentro de la cadena que marca el lugar inmediatamente después de la subcadena devuelta.

Descripción

Devuelve una subcadena de la cadena especificada antes de la primera ocurrencia del patrón especificado.

- Si **<string>** no contiene la subcadena **<pattern>**, se devuelve una cadena vacía.
- Si **<pattern>** es una cadena vacía, se devuelve **<string>**.

- Si `<string>` o `<pattern>` es NULL, entonces se devuelve NULL.

Ejemplo

El siguiente ejemplo devuelve `He llo`, la parte de la cadena indicada que se encuentra a la izquierda de la primera ocurrencia de `My`:

```
SELECT SUBSTR_BEFORE ('Hello My Friend', 'My') "substr before" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función SUBSTRING (cadena)

Devuelve una subcadena de un valor de entrada, empezando por una posición especificada dentro del valor de entrada.

Sintaxis

```
SUBSTRING(<string>, <start_position> [, <string_length>])
```

Elementos de sintaxis

`<string>`

Especifica la cadena que contiene la subcadena. `<string>` también puede ser de tipo binario BLOB o VARBINARY.

`<start_position>`

Especifica dónde empieza la subcadena dentro de la cadena.

`<string_length>`

Especifica la longitud de la subcadena.

Descripción

Devuelve una subcadena de la cadena especificada empezando por `<start_position>` dentro de la cadena. SUBSTRING puede devolver la parte restante de una cadena desde `<start_position>`, u opcionalmente, un número de caracteres establecido por el parámetro `<string_length>`.

- Si `<start_position>` es menor que 1, se considera que es 1.
- Si `<string_length>` es menor que 1, se devuelve una cadena vacía.
- Si `<string_length>` es mayor que la longitud de la parte restante de `<str>`, la parte restante se devuelve sin relleno en blanco.

Cuando se utiliza en tipos binarios, SUBSTRING determina la longitud de bytes e interpreta los desplazamientos, `<start_position>` y `<string_length>`, como posiciones de bytes.

Ejemplo

El siguiente ejemplo selecciona dos caracteres de la cadena `1234567890` empezando en la posición 4 y devuelve el valor `45`:

```
SELECT SUBSTRING ( '1234567890',4,2) "substring" FROM DUMMY;
```

El siguiente ejemplo devuelve 'ABCD' del valor binario x'ABCDEF':

```
SELECT SUBSTRING(x'ABCDEF',1,2) "substring" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función SUBSTRING_REGEXPR (cadena)

Busca un patrón de expresión regular en una cadena y devuelve una ocurrencia de la subcadena coincidente.

Sintaxis

```
SUBSTR[ING]_REGEXPR( <pattern> [ FLAG <flag> ] IN <regex_subject_string>
[ FROM <start_position> ]
[ OCCURRENCE <regex_occurrence> ]
[ GROUP <regex_capture_group> ] )
```

Elementos de sintaxis

<pattern>

Especifica un patrón de búsqueda basado en Perl Compatible Regular Expression (PCRE).

```
<pattern> ::= !!Perl Compatible Regular Expression
```

<flag>

Especifica que el comportamiento coincidente de la función se puede definir mediante el literal de indicador. Están disponibles las siguientes opciones:

Marcar opciones

Marcar opción	Descripción
i	Activa la coincidencia que no distingue entre mayúsculas y minúsculas
m	Activa el modo multilínea, donde <subject_string> se tratará como varias líneas y la expresión ^ y \$ coinciden justo después o justo antes, respectivamente, de un terminador de línea o el final de la secuencia de entrada
s	Habilita la expresión <.> como comodín para que coincida con cualquier carácter, incluido un terminador de línea
x	Permite espacios en blanco y comentarios en el patrón
U	SAP HANA utiliza la biblioteca Expresiones regulares compatibles con Perl (PCRE) para procesar expresiones regulares. Al especificar 'U' (abreviatura de "poco codicioso") se invierte la "codicia" de los cuantificadores para que no sean codiciosos por defecto sino que se vuelvan codiciosos solo cuando vayan seguidos de "?". La coincidencia poco codiciosa a menudo puede funcionar más rápido porque encuentra la coincidencia más corta en momentos en los que solo es interesante saber si hay alguna coincidencia. Para una comprensión completa del comportamiento de coincidencia "codicioso" frente a "poco codicioso" de la biblioteca de expresiones regulares compatibles con Perl (PCRE), visite:https://www.pcre.org/original/doc/html/pcrematching.html .

```
<flag> ::= 'i' | 'm' | 's' | 'x'
```

<regex_subject_string>

Define la cadena a la que se debe aplicar el patrón de búsqueda. Si <regex_subject_string> está vacío, el resultado estará vacío.

```
<regex_subject_string> ::= <string>
```

<start_position>

Fijar este parámetro en un entero positivo indica en qué carácter de <regex_subject_string> empieza la búsqueda. Si <start_position> no es un entero positivo, entonces se devuelve NULL.

```
<start_position> ::= <numeric_literal>
```

<regex_occurrence>

Fijar este parámetro en un entero positivo indica la ocurrencia del <pattern> en <regex_subject_string>. El valor predeterminado es 1 y devuelve la primera ocurrencia. Si <regex_aparece> no es un entero positivo, entonces se devuelve NULL.

```
<regex_occurrence> ::= <numeric_literal >
```

<regex_capture_group>

Indica el número del grupo de subcadenas capturado por la expresión regular. El valor predeterminado es 0. Si <regex_capture_group> es un entero negativo, entonces se devuelve 0.

```
<regex_capture_group> ::= <integer>
```

Descripción

Busca un patrón de expresión regular en una cadena y devuelve una ocurrencia de la subcadena coincidente.

Si alguno de los siguientes parámetros es NULL, la función devuelve NULL: <pattern>, <flag>, <regex_subject_string>, <start_position>, <regex_luck> o <regex_capture_group>.

Ejemplo

El siguiente ejemplo devuelve el día 01 del valor de fecha 20140401:

```
SELECT SUBSTR_REGEXPR('([[:digit:]]{4})([[:digit:]]{2})([[:digit:]]{2})' IN '20140401' GROUP 3) "s
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función SUBVECTOR (vector)

Devuelve un subvector de un vector.

Sintaxis

```
SUBVECTOR(<vector>, <start_position>, <vector_dimension>)
```

Elementos de sintaxis

<vector>

Especifica el vector.

<start_position>

Especifica dónde empieza el subvector dentro del vector.

<vector_dimension>

Especifica la dimensión de subvector.

Descripción

Devuelve un subvector de <vector> que empieza por <start_position> dentro de <vector>.

- Si <start_position> es menor que 1, se considera que es 1.
- Si <start_position> es mayor que la dimensión del vector, entonces se devuelve NULL.
- Si <vector_dimension> es mayor que la parte restante de <vector>, entonces se devuelve la parte restante de <vector>.
- Si <vector_dimension> es menor que 1, entonces se devuelve la parte restante de <vector>.

Ejemplos

El siguiente ejemplo devuelve [3,4,5,6]:

```
SELECT
  TO_ARRAY(SUBVECTOR(TO_REAL_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), 3, 4))
FROM DUMMY;

SELECT
  TO_ARRAY(SUBVECTOR(TO_HALF_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), 3, 4))
FROM DUMMY;
```

El siguiente ejemplo devuelve [1,2,3]:

```
SELECT
  TO_ARRAY(SUBVECTOR(TO_REAL_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), -1, 3))
FROM DUMMY;

SELECT
  TO_ARRAY(SUBVECTOR(TO_HALF_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), -1, 3))
FROM DUMMY;
```

El siguiente ejemplo devuelve [5,6,7,8,9,10]:

```
SELECT
  TO_ARRAY(SUBVECTOR(TO_REAL_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), 5, 10))
FROM DUMMY;

SELECT
  TO_ARRAY(SUBVECTOR(TO_HALF_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), 5, 10))
FROM DUMMY;
```

El siguiente ejemplo devuelve [5,6,7,8,9,10]:

```
SELECT
  TO_ARRAY(SUBVECTOR(TO_REAL_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), 5, -100))
```

```
FROM DUMMY;

SELECT
    TO_ARRAY(SUBVECTOR(TO_HALF_VECTOR(' [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] '), 5, -100))
FROM DUMMY;
```

En comparación con los elementos REAL_VECTOR, los elementos HALF_VECTOR tienen una precisión reducida y un rango de valores reducido. Esto puede provocar resultados inesperados:

```
SELECT
    TO_NVARCHAR(SUBVECTOR(TO_REAL_VECTOR(' [33333] '), 1, 1)),
    TO_NVARCHAR(SUBVECTOR(TO_HALF_VECTOR(' [33333] '), 1, 1))
FROM DUMMY;

returns:
[33333]    [33344]
```

Información relacionada

[Tipos de datos vectoriales](#)

Función SUM (agregar)

Devuelve la suma de la expresión. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
SUM( [ ALL | DISTINCT ] <expression> )
```

Función de ventana:

```
SUM( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica los datos de entrada para la función.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada

TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL(p, s)	DECIMAL	REAL	DOUBLE
DECIMAL(9,6)	DECIMAL(11,6)	DECIMAL(16,6)	DECIMAL(25,6)	DECIMAL(p', s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- $p' = 18$, si $p \leq 18$
- $p' = 28$, si $18 < p \leq 28$
- $p' = 38$, si $28 < p \leq 38$

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";
CREATE COLUMN TABLE "MyProducts" (
  "Product_ID" NVARCHAR(10),
  "Product_Name" NVARCHAR(100),
  "Category" NVARCHAR(100),
  "Quantity" INTEGER,
  "Price" DECIMAL(10,2),
  PRIMARY KEY ("Product_ID") );

INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32, 20.99);
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);
INSERT INTO "MyProducts" VALUES('P4','Coats', 'Clothes', 5, 129.99);
INSERT INTO "MyProducts" VALUES('P5','Purse', 'Accessories', 3, 89.49);
```

El siguiente ejemplo suma las cantidades de abrigos y chaquetas y devuelve el valor 21:

```
SELECT SUM("Quantity") FROM "MyProducts" WHERE "Product_Name" IN ('Jackets', 'Coats');
```

Información relacionada

[Funciones de ventana y la especificación de ventana](#)

[Funciones agregadas de ventana](#)

[Agregar funciones](#)

[Expresiones](#)

Función SYSUUID (varios)

Devuelve un nuevo identificador único universal generado por la instancia de SAP HANA conectada.

Sintaxis

SYSUUID

Descripción

Cada vez que llama la función SYSUUID, devuelve un nuevo valor UUID. Las llamadas SYSUUID de varias conexiones se serializan internamente para garantizar la generación de valores unívocos.

Ejemplo

La siguiente consulta devuelve el valor generado de forma unívoca CF773C584E3A6213E2008AC320F0F2BE:

```
SELECT SYSUUID FROM DUMMY;
```

Función TAN (numérica)

Devuelve la tangente de un número especificado, donde el argumento es un ángulo expresado en radianes.

Sintaxis

TAN(<number>)

Descripción

Devuelve la tangente de <número>, donde <n> es un ángulo expresado en radianes.

Ejemplo

El siguiente ejemplo devuelve el valor 0 de "tan":

```
SELECT TAN (0.0) "tan" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función TANH (numérica)

Devuelve la tangente hiperbólica del argumento numérico especificado.

Sintaxis

TANH(<number>)

Descripción

Devuelve la tangente hiperbólica del argumento numérico <número>.

Ejemplo

El siguiente ejemplo devuelve el valor 0,7615941559557649 para "tanh":

```
SELECT TANH (1.0) "tanh" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función TO_ARRAY (conversión de tipo de datos)

Convierte un vector en un tipo de datos ARRAY.

Sintaxis

```
TO_ARRAY(<vector>)
```

Elementos de sintaxis

<vector>

Especifica el vector que se va a convertir.

Descripción

Convierte <vector> en un tipo de datos ARRAY. La matriz contiene los mismos elementos que el vector pasado a la función.

Ejemplos

Los siguientes ejemplos devuelven la siguiente matriz (1,2,,3):

```
SELECT TO_ARRAY(TO_REAL_VECTOR(' [1,2,3] ')) FROM DUMMY;
```

```
SELECT TO_ARRAY(TO_HALF_VECTOR(' [1,2,3] ')) FROM DUMMY;
```

Información relacionada

[Función TO_BLOB \(conversión de tipo de datos\)](#)

[Función TO_NVARCHAR \(conversión de tipo de datos\)](#)

[Función TO_REAL_VECTOR \(conversión de tipo de datos\)](#)

Función TO_BIGINT (conversión de tipo de datos)

Convierte un valor en un tipo de datos BIGINT.

Sintaxis

```
TO_BIGINT(<value>)
```

Descripción

Si la entrada *<valor>* tiene una mantisa, estos dígitos se truncan durante el proceso de conversión.

Ejemplos

El siguiente ejemplo convierte el valor 10 en un valor BIGINT 10:

```
SELECT TO_BIGINT ('10') "to bigint" FROM DUMMY;
```

El siguiente ejemplo convierte el valor 10.5 en un valor BIGINT 10, que trunca la mantisa:

```
SELECT TO_BIGINT (10.5) "to bigint" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_BINARY (conversión de tipo de datos)

Convierte un valor en un tipo de datos BINARY.

Sintaxis

```
TO_BINARY(<value>)
```

Descripción

Convierte un *<valor>* en un tipo de datos BINARY.

Ejemplo

En el siguiente ejemplo, se convierte el valor abc al valor BINARY 616263.

```
SELECT TO_BINARY ('abc') "to binary" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_BLOB (conversión de tipo de datos)

Convierte un string binario, NCLOB (o CLOB que es un alias de NCLOB) o un tipo de datos vectorial en un tipo de datos BLOB.

Sintaxis

TO_BLOB(<value>)

<value> ::= <binary string> | <nclob_value> | <clob_value> | <vector>

Descripción

Convierte un BINARY, NCLOB (o CLOB que es un alias de NCLOB) o un vector <valor> en un tipo de datos BLOB.

Ejemplo

El siguiente ejemplo convierte el valor BINARY abcde en el valor BLOB abcde:

```
SELECT TO_BLOB (TO_BINARY('abcde')) "to blob" FROM DUMMY;
```

El siguiente ejemplo convierte el valor NCLOB abc en el valor BLOB abc:

```
SELECT TO_BLOB (TO_NCLOB('abc')) "to blob" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_BOOLEAN (conversión de tipo de datos)

Convierte un valor en un tipo de datos BOOLEAN.

Sintaxis

TO_BOOLEAN(<value>)

Elementos de sintaxis

<value>

Si <valor> es:

- 1, 'true' o true y TO_BOOLEAN devuelve el valor true.
- 0, 'false' o false y TO_BOOLEAN devuelve el valor false.
- desconocido (booleano) o 'desconocido', TO_BOOLEAN devuelve el valor desconocido.

"verdadero", "falso" y "desconocido" no distinguen entre mayúsculas y minúsculas, lo que significa que los valores de "TRUE" y "TruE" se tratan igual que un valor de verdadero. TO_BOOLEAN lanza una excepción cuando <valor> no es verdadero, falso, desconocido, 0, 1, '0', '1', 'verdadero', 'falso' o 'desconocido'.

Descripción

Convierte un <valor> en un tipo de datos BOOLEAN.

Ejemplo

Convierte el valor 0 en el valor BOOLEAN FALSE.

```
SELECT TO_BOOLEAN(0) FROM DUMMY;
```

Convierte el valor 1 en el valor BOOLEAN TRUE.

```
SELECT TO_BOOLEAN(1) FROM DUMMY;
```

Información relacionada

[Tipo de datos booleano](#)

Función TO_CLOB (conversión de tipo de datos)

Convierte un valor en un tipo de datos NCLOB. En su lugar, utilice la función TO_NCLOB.

Sintaxis

```
TO_CLOB(<value>)
```

Elementos de sintaxis

Consulte la función TO_NCLOB para la descripción de <value>.

Descripción

Mientras que la función se llama TO_CLOB, en SAP HANA CLOB es un alias para el tipo de datos NCLOB. En su lugar, utilice la función TO_NCLOB.

Información relacionada

[Función TO_NCLOB \(conversión de tipo de datos\)](#)

[Conversión de tipo de datos](#)

Función TO_DATE (conversión de tipo de datos)

Convierte una cadena de fecha en un tipo de datos DAYDATE.

Sintaxis

```
TO_DATE(<date> [, <format>])
```

Descripción

Convierte la cadena de fecha <date> en un tipo de datos DAYDATE.

Si se omite el especificador *<format>*, la conversión se realiza mediante el modelo de formato de fecha.

Ejemplo

El siguiente ejemplo convierte la cadena 2010-01-12 en un valor DAYDATE con el formato AAAA-MM-DD y devuelve el valor 2010-01-12 (u otro formato como 12 de enero de 2010, dependiendo de las opciones de visualización de fecha):

```
SELECT TO_DATE('2010-01-12', 'YYYY-MM-DD') "to date" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_DATS (conversión de tipo de datos)

Convierte una cadena de fecha en una cadena DATE ABAP.

Sintaxis

```
TO_DATS(<date>)
```

Descripción

Convierte el string de fecha *<date>* en un string DATE ABAP con el formato "AAAAMMDD".

Ejemplo

El siguiente ejemplo convierte el valor 2010-01-12 en el string ABAP DATE 20100112.

```
SELECT TO_DATS ('2010-01-12') "abap date" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_DECIMAL (conversión de tipo de datos)

Convierte un valor en un tipo de datos DECIMAL.

Sintaxis

```
TO_DECIMAL(<value> [, <precision>, <scale>])
```

Elementos de sintaxis

<value>

Especifica el valor que se va a convertir en un tipo de datos DECIMAL. *<valor>* puede ser una variable.

<precision>

Especifica el número total de dígitos significativos. *<precisión>* puede oscilar entre 1 y 38, especificado como una constante de cadena. Si no se especifica *<precisión>*, 34 es el valor predeterminado.

<scale>

Especifica el número de dígitos desde la coma decimal hasta el dígito menos significativo. *<escala>*. *<escala>* debe estar entre 0 y el valor de *<precisión>*. La escala es positiva cuando el número tiene dígitos significativos a la derecha de la coma decimal, y negativo cuando el número tiene dígitos significativos a la izquierda de la coma decimal.

Descripción

Si no se especifica la escala, el valor predeterminado es 0. Si no se especifican *<precisión>* y *<escala>*, DECIMAL se convierte en un número decimal de coma flotante, que cubre de 1 a 34 para la precisión y de -6,111 a 6,176 para la escala, dependiendo del *<valor>*.

Los dígitos menos significativos innecesarios en la mantisa del valor de entrada se truncan durante el proceso de conversión.

Ejemplo

En el siguiente ejemplo, se convierte el valor 7654321.888888 a DECIMAL con una precisión de 10 dígitos y una escala de 3, y se muestra el valor 7654321,888:

```
SELECT TO_DECIMAL(7654321.888888, 10, 3) "to decimal" FROM DUMMY;
```

El siguiente ejemplo convierte el valor 1234.789 en DECIMAL con 10 para la precisión y 0 para la escala y devuelve el valor 1234 (la escala se define como 0, por lo que todo se trunca después de la precisión).

```
SELECT TO_DECIMAL(1234.789,10,0) AS "to decimal" FROM DUMMY;
```

En el siguiente ejemplo, convierte el valor 12345678901234567812345678901256.78 en un tipo de datos DECIMAL, con 34 como precisión (predeterminada) y 0 como escala (predeterminada), y devuelve 1234567890123456789012345678901256 (los últimos dos dígitos se truncan porque la precisión predeterminada es 34).

```
SELECT TO_DECIMAL(1234567890123456789012345678901256.78) "to decimal" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

[Tipos de datos numéricos](#)

Función TO_DOUBLE (conversión de tipo de datos)

Convierte un valor en un tipo de datos DOUBLE.

Sintaxis

```
TO_DOUBLE(<value>)
```


Descripción

Convierte un *<valor>* especificado en un tipo de datos DOUBLE (doble precisión).

Ejemplo

El siguiente ejemplo convierte 15.12 en un DOUBLE y, a continuación, multiplica el valor por 3, devolviendo el valor DOUBLE 45.36.

```
SELECT 3*TO_DOUBLE ('15.12') "to double" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_HALF_VECTOR (conversión de tipo de datos)

Construye un vector a partir de una representación textual o binaria, a partir de una matriz o un vector con el tipo de datos REAL_VECTOR.

Sintaxis

```
TO_HALF_VECTOR(<vector_representation>)
```

Elementos de sintaxis

<vector_representation>

Una representación vectorial admitida, que puede ser una representación textual o binaria, o una matriz con elementos numéricos.

Descripción

Construye y devuelve un vector con elementos de *<representación_vector>* que se almacenan internamente con media precisión.

Si *<representación_vector>* es una representación textual, se espera el siguiente formato:

```
'[' <number> ( ',' <number> )* ']'  
for example:  
[0.342, 0.163, 0.426]
```

Si *<representación_vector>* es una representación binaria, se espera que los datos comiencen con un entero de 32 bits little-endian *n* que describe la dimensión del vector seguido de *n* pequeños endian IEEE 754 números de punto flotante de media precisión que representan los elementos vectoriales. Una representación codificada hexadecimal podría tener el siguiente aspecto: 03000000 003C 0040 0042.

Si *<representación_vector>* es una matriz, se espera que los miembros de la matriz sean números y que todos los números no sean NULL.

La dimensión mínima de un vector es 1. La dimensión máxima es 65.000.

Ejemplo

Los siguientes ejemplos muestran cómo construir el vector [1, 2, 3] a partir de una representación textual, binaria y de matriz:

```
SELECT TO_HALF_VECTOR(' [1,2,3] ') FROM DUMMY;  
SELECT TO_HALF_VECTOR(x'030000000003C00400042') FROM DUMMY;  
SELECT TO_HALF_VECTOR(ARRAY(1,2,3)) FROM DUMMY;
```

TO_HALF_VECTOR puede emitir valores REAL_VECTOR, pero puede provocar desbordamientos numéricos:

```
-- works fine  
SELECT TO_REAL_VECTOR(' [1e32] ') FROM DUMMY;  
  
-- throws an exception  
SELECT TO_HALF_VECTOR(TO_REAL_VECTOR(' [1e32] ')) FROM DUMMY;
```

Información relacionada

[Función TO_ARRAY \(conversión de tipo de datos\)](#)

[Función TO_BLOB \(conversión de tipo de datos\)](#)

[Función TO_NVARCHAR \(conversión de tipo de datos\)](#)

Función TO_INT (conversión de tipo de datos)

Convierte un valor en un tipo de datos INT.

Sintaxis

TO_INT(<value>)

Descripción

Si la entrada <valor> tiene una mantisa, entonces la mantisa se trunca durante el proceso de conversión.

Ejemplos

En el siguiente ejemplo, se convierte el valor 10 al valor INT 10:

```
SELECT TO_INT('10') "to int" FROM DUMMY;
```

El siguiente ejemplo convierte el valor 10.5 en el valor INT 10, que trunca la mantisa:

```
SELECT TO_INT(10.5) "to int" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_INTEGER (conversión de tipo de datos)

Convierte el <valor> en un tipo de datos INTEGER.

Sintaxis

```
TO_INTEGER(<value>)
```

Descripción

Si la entrada <valor> tiene una mantisa, estos dígitos se truncan durante el proceso de conversión.

Ejemplos

El siguiente ejemplo convierte el valor 10 en el valor INTEGER 10:

```
SELECT TO_INTEGER ('10') "to int" FROM DUMMY;
```

El siguiente ejemplo convierte el valor 10.5 en el valor INTEGER 10, que trunca la mantisa:

```
SELECT TO_INTEGER(10.5) "to int" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

TO_JSON_BOOLEAN (conversión de tipo de datos)

Convierte un <valor> dado en un valor booleano en formato JSON para utilizarlo con la función Almacén de documentos JSON.

Sintaxis

```
TO_JSON_BOOLEAN(<value>)
```

Descripción

Si el <valor> no se puede convertir en un valor booleano JSON, se devuelve un error.

Ejemplo

Cree y rellene la colección myCollection1:

i Nota

Consulte [la sentencia CREATE COLLECTION](#) para obtener ayuda sobre la creación de colecciones.

```
CREATE COLLECTION myCollection1;  
INSERT INTO myCollection1 VALUES ('{"k1" : true}');
```

La ejecución de la siguiente sentencia devuelve un error:

```
SELECT * FROM myCollection1 WHERE "k1" = TRUE;
```

La ejecución de la siguiente sentencia devuelve el registro:

```
SELECT * FROM myCollection1 WHERE "k1" = TO_JSON_BOOLEAN(TRUE);
```

Función TO_NCLOB (conversión de tipo de datos)

Convierte un valor al tipo de datos NCLOB.

Sintaxis

```
TO_NCLOB(<value>)
```

Descripción

Convierte un <valor> en un tipo de datos NCLOB.

Ejemplo

El siguiente ejemplo convierte el valor `TO_NCLOB` convierte el valor en un tipo de datos NCLOB en un valor NCLOB.

```
SELECT TO_NCLOB ('TO_NCLOB converts the value to a NCLOB data type') "to nclob" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_NVARCHAR (conversión de tipo de datos)

Convierte un valor indicado en un tipo de datos NVARCHAR, con una opción para dar formato al valor de salida.

Sintaxis

```
TO_NVARCHAR( <value> [ , <format> ] )
```

Elementos de sintaxis

<value>

El valor de entrada para la función. Si <value> es un tipo de datos de fecha y hora y no se especifica <format>, el formato predeterminado se aplica al valor de salida, a menos que la variable de sesión relevante (DATE_FORMAT, TIME_FORMAT, TIMESTAMP_FORMAT o SECONDDATE_FORMAT) se fije en algo diferente.

<format>

Especifica el formato deseado que se aplicará al valor convertido. Para los valores de fecha y hora, utilice <formato> para definir el formato de salida de fecha/hora deseado (por ejemplo, AAAA/MM/DD. Consulte el tema *Tipos de datos de fecha y hora* de esta guía para obtener más información sobre los posibles formatos de fecha y hora.

Para definir otros tipos de formatos de salida (por ejemplo, números con símbolos personalizados, comas, ceros, etc.), especifique *<formato>* utilizando las siguientes convenciones:

- 9 - Devuelve el número en la posición especificada; de lo contrario, no devuelve nada.
- 0 - Devuelve el número en la posición especificada; de lo contrario, devuelve un cero (0).
- S: devuelve el símbolo de signo (ya sea + o -) para el valor.
- E - Divida el número en parte significativa y parte de exponente.
- % - Multiplicar *<valor>* por 10² y añada un símbolo de porcentaje (%) al final.
- . (un período) - Inserte un período en la posición especificada.
- Todos los demás caracteres que no sean los elementos anteriores: Devuelve el carácter en la posición especificada.

Descripción

Si se omite el especificador *<format>*, la conversión se realiza mediante el modelo de formato de fecha.

Los siguientes tipos de datos se pueden convertir a NVARCHAR mediante la función TO_NVARCHAR:

- BIGINT, DAYDATE, DECIMAL, DOUBLE, FIXED12, FIXED16, FIXED8, INTEGER, REAL, SECONDDATE, SMALLDECIMAL, SMALLINT, TIME, TIMESTAMP, TINYINT, VARBINARY, VARCHAR
- CLOB, NCLOB
- REAL_VECTOR, HALF_VECTOR

Ejemplo

En el siguiente ejemplo, se convierte el valor 2009/12/31 al valor NVARCHAR 09-12-31.

```
SELECT TO_NVARCHAR(TO_DATE('2009/12/31'), 'YY-MM-DD') "to nvarchar" FROM DUMMY;
```

Las siguientes sentencias demuestran cómo utilizar la opción *<string_format>* para definir el formato de los valores de salida:

```
SELECT TO_NVARCHAR(1, '00.00') FROM Dummy;    --> 01.00
SELECT TO_NVARCHAR(100, '00.00') FROM Dummy;    --> 100.00
SELECT TO_NVARCHAR(100, '9999.00') FROM Dummy;    --> 100.00
SELECT TO_NVARCHAR(100, '0000.00') FROM Dummy;    --> 0100.00
SELECT TO_NVARCHAR(100, 'S0000.00') FROM Dummy;    --> +0100.00
SELECT TO_NVARCHAR(-100, 'S0000.00') FROM Dummy;    --> -0100.00
SELECT TO_NVARCHAR(-100, '000.00') FROM Dummy;    --> -100.00
SELECT TO_NVARCHAR(-100, 'S0.0E0') FROM Dummy;    --> -1.0E2
SELECT TO_NVARCHAR(-0.001, 'S0.0E0') FROM Dummy;    --> -1.0E-3
SELECT TO_NVARCHAR(-0.001, 'S0.0E00') FROM Dummy;    --> -1.0E-03
SELECT TO_NVARCHAR(1000, '9,999.00') FROM Dummy;    --> 1,000.00
SELECT TO_NVARCHAR(1000, '$9,999.00') FROM Dummy;    --> $1,000.00
SELECT TO_NVARCHAR(1000, '$9,999.99') FROM Dummy;    --> $1,000.
SELECT TO_NVARCHAR(1, '0.00%') FROM Dummy;    --> 100.00%
SELECT TO_NVARCHAR(1, '100%') FROM Dummy;    --> 1100%
```

Información relacionada

[Tipos de datos de fecha y hora](#)[Función TO_VARCHAR \(conversión de tipo de datos\)](#)[Conversión de tipo de datos](#)

Función TO_REAL (conversión de tipo de datos)

Convierte un <valor> en un tipo de datos REAL.

Sintaxis

TO_REAL(<value>)

Descripción

Convierte un <valor> en un tipo de datos REAL (precisión individual).

Ejemplo

El siguiente ejemplo convierte el valor 15 . 12 en un valor REAL y lo multiplica por 3 para devolver el valor 45 . 36000061035156.

```
SELECT 3*TO_REAL ('15.12') "to real" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_REAL_VECTOR (conversión de tipo de datos)

Construye un vector a partir de una representación textual o binaria, a partir de una matriz o un vector con el tipo de datos HALF_VECTOR.

Sintaxis

TO_REAL_VECTOR(<vector_representation>)

Elementos de sintaxis

<vector_representation>

Una representación vectorial admitida, que puede ser una representación textual o binaria, o una matriz con elementos numéricos.

Descripción

Construye y devuelve un vector con elementos REAL de <representación_vector>.

Si <representación_vector> es una representación textual, se espera el siguiente formato:

```
'[' <number> ( ' , ' <number> ) * ' ] '
for example:
[0.342, 0.163, 0.426]
```

Si *<representación_vector>* es una representación binaria, se espera que los datos comiencen con un entero de 32 bits little-endian *n* que describe la dimensión del vector seguido de *n* pequeños endian IEEE 754 números de coma flotante de precisión única que representan los elementos vectoriales. Una representación codificada hexadecimal podría tener el siguiente aspecto:

03000000 0000803F 00000040 00004040.

Si *<representación_vector>* es una matriz, se espera que los miembros de la matriz sean números y que todos los números no sean NULL.

La dimensión mínima de un vector es 1. La dimensión máxima es 65.000.

Ejemplo

El siguiente ejemplo muestra cómo construir el vector [1, 2, 3] a partir de una representación textual, binaria y de matriz:

```
SELECT TO_REAL_VECTOR(' [1,2,3] ') FROM DUMMY;
SELECT TO_REAL_VECTOR(x'03000000000000803F0000004000004040') FROM DUMMY;
SELECT TO_REAL_VECTOR(ARRAY(1,2,3)) FROM DUMMY;
SELECT TO_REAL_VECTOR(TO_HALF_VECTOR(' [1,2,3] ')) FROM DUMMY;
```

Información relacionada

[Función TO_ARRAY \(conversión de tipo de datos\)](#)

[Función TO_BLOB \(conversión de tipo de datos\)](#)

[Función TO_NVARCHAR \(conversión de tipo de datos\)](#)

Función TO_SECONDDATE (conversión de tipo de datos)

Convierte una cadena de fecha especificada en un tipo de datos SECONDDATE.

Sintaxis

```
TO_SECONDDATE(<date> [, <format>])
```

Descripción

Si se omite el especificador *<format>*, la conversión se realiza mediante el modelo de formato de fecha.

Ejemplo

En el siguiente ejemplo, se convierte el valor 2010-01-11 13:30:00 a un tipo de datos SECONDDATE con el formato YYYY-MM-DD HH24:MI:SS y se muestra el valor 2010-01-11 13:30:00.0 (u otro formato como Jan 11, 2010 1:30:00.0 PM, según la configuración de visualización de la fecha):

```
SELECT TO_SECONDDATE ('2010-01-11 13:30:00', 'YYYY-MM-DD HH24:MI:SS') "to seconddate" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_SMALLDECIMAL (conversión de tipo de datos)

Convierte el valor especificado en un tipo de datos SMALLDECIMAL.

Sintaxis

```
TO_SMALLDECIMAL(<value>)
```

Descripción

Convierte el valor especificado en un tipo de datos SMALLDECIMAL.

Ejemplo

En el siguiente ejemplo, se convierte el valor 7654321.89 al valor SMALLDECIMAL 7,654,321.89:

```
SELECT TO_SMALLDECIMAL(7654321.89) "to smalldecimal" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_SMALLINT (conversión de tipo de datos)

Convierte un valor en un tipo de datos SMALLINT.

Sintaxis

```
TO_SMALLINT(<value>)
```

Descripción

Si la entrada <valor> tiene una mantisa, estos dígitos se truncan durante el proceso de conversión.

Ejemplos

En el siguiente ejemplo, se convierte el valor 10 a SMALLINT y se muestra el valor 10:

```
SELECT TO_SMALLINT ('10') "to smallint" FROM DUMMY;
```

El siguiente ejemplo convierte el valor 10 en SMALLINT y devuelve el valor 10, que trunca la mantisa:

```
SELECT TO_SMALLINT(10.5) "to int" FROM DUMMY;
```


Información relacionada

[Conversión de tipo de datos](#)

Función TO_TIME (conversión de tipo de datos)

Convierte una cadena de hora especificada en un tipo de datos TIME.

Sintaxis

```
TO_TIME(<time> [, <format>])
```

Descripción

Convierte la cadena de tiempo <time> en el tipo de datos TIME.

Si se omite el especificador <format>, la conversión se realiza mediante el modelo de formato de tiempo.

Ejemplo

El siguiente ejemplo convierte el valor 08:30 AM en un valor TIME con el formato HH:MI AM y devuelve el valor 08:30:00:

```
SELECT TO_TIME ('08:30 AM', 'HH:MI AM') "to time" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_TIMESTAMP (conversión de tipo de datos)

Convierte una cadena de fecha en un tipo de datos TIMESTAMP.

Sintaxis

```
TO_TIMESTAMP(<date> [, <format>])
```

Descripción

Convierte la cadena de fecha <date> al tipo de datos TIMESTAMP.

Si se omite el especificador <format>, la conversión se realiza mediante el modelo de formato de fecha.

Ejemplo

En el siguiente ejemplo, se convierte el valor 11-ENan-2010 01:30 PM al valor TIMESTAMP 2010-01-11 13:30:00.0 con el formato DD-Mon-YYYY HH:MI AM:

```
SELECT TO_TIMESTAMP ('11-Jan-2010 01:30 PM', 'DD-Mon-YYYY HH:MI AM') AS "to timestamp" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_TINYINT (conversión de tipo de datos)

Convierte un valor en un tipo de datos TINYINT.

Sintaxis

```
TO_TINYINT(<value>)
```

Descripción

Convierte el <valor> en un tipo de datos TINYINT.

Si el valor de entrada tiene una mantisa, estos dígitos se truncan durante el proceso de conversión.

Ejemplos

El siguiente ejemplo convierte el valor 10 en el valor TINYINT 10:

```
SELECT TO_TINYINT ('10') "to tinyint" FROM DUMMY;
```

El siguiente ejemplo convierte el valor 10.5 en el valor TINYINT 10, que trunca la mantisa:

```
SELECT TO_TINYINT(10.5) "to tinyint" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

Función TO_VARBINARY (conversión de tipo de datos)

Convierte un valor en un tipo de datos BINARY.

Sintaxis

```
TO_VARBINARY(<value>)
```

Descripción

Convierte un <valor> en un tipo de datos BINARY.

Ejemplo

En el siguiente ejemplo, se convierte el valor `abc` al valor BINARY 646566.

```
SELECT VARBINARY ('def') "to binary" FROM DUMMY;
```

Información relacionada

[Conversión de tipo de datos](#)

[Tipos de datos binarios](#)

Función TO_VARCHAR (conversión de tipo de datos)

En la base de datos SAP HANA, VARCHAR es un alias para el tipo de datos NVARCHAR. En su lugar, utilice la función TO_NVARCHAR.

Sintaxis

```
TO_VARCHAR(<value> [, <format>])
```

Elementos de sintaxis

Consulte la función TO_NVARCHAR para las descripciones de *<valor>* y *<formato>*.

Descripción

Convierte un valor indicado en un tipo de datos VARCHAR, con una opción para dar formato al valor de salida. Mientras que la función se llama TO_VARCHAR, en SAP HANA, VARCHAR es un alias para el tipo de datos NVARCHAR. Por lo tanto, utilice la función TO_NVARCHAR en su lugar.

Información relacionada

[Función TO_NVARCHAR \(conversión de tipo de datos\)](#)

[Tipos de datos de fecha y hora](#)

[Conversión de tipo de datos](#)

Función TRIM (cadena)

Devuelve una cadena después de eliminar los espacios iniciales y finales.

Sintaxis

```
TRIM([[LEADING | TRAILING | BOTH] <trim_char> FROM] <string> )
```

Elementos de sintaxis

<trim_char>

Especifica los caracteres que se deben eliminar de una cadena.

Descripción

Devuelve la cadena *<string>* después de eliminar los espacios iniciales y finales. La operación de recorte se lleva a cabo desde el inicio (LEADING), el final (TRAILING) o ambos (BOTH) extremos de la cadena.

- Si *<string>* o *<trim_char>* son valores NULL, se devuelve NULL.
- Si no se especifica ninguna opción, TRIM elimina la subcadena inicial y final *<trim_char>* de la cadena *<cadena>*.
- Si no se especifica *<trim_char>*, se utiliza un único espacio en blanco.

Ejemplo

El siguiente ejemplo elimina el carácter a tanto al principio como al final de la cadena especificada y devuelve el valor 123456789:

```
SELECT TRIM ('a' FROM 'aaa123456789aa') "trim both" FROM DUMMY;
```

El siguiente ejemplo elimina el carácter a al principio de la cadena especificada y devuelve el valor 123456789aa:

```
SELECT TRIM (LEADING 'a' FROM 'aaa123456789aa') "trim leading" FROM DUMMY;
```

Función TRIM_ARRAY (matriz)

Elimina el número especificado de elementos del final de una matriz.

Sintaxis

```
TRIM_ARRAY(<array_value_expression>, <truncate_length>)
```

Elementos de sintaxis

<array_value_expression>

Especifica la matriz de la que la función elimina los elementos.

<truncate_length>

Especifica el número de elementos que se van a eliminar al final de la matriz.

Descripción

Devuelve una matriz que tiene un número especificado de elementos recortados del final.

Si *<truncate_length>* es menor o igual que 0, se genera una excepción.

Ejemplo

El siguiente ejemplo demuestra la eliminación de tres elementos de una matriz. La consulta devuelve 10.

```
CREATE COLUMN TABLE ARRAY_TEST (IDX INT, VAL INT ARRAY);  
INSERT INTO ARRAY_TEST VALUES (1, ARRAY(1, 2, 3));  
INSERT INTO ARRAY_TEST VALUES (2, ARRAY(10, 20, 30, 40));  
  
SELECT TRIM_ARRAY(VAL, 3) "trim_array" FROM ARRAY_TEST;
```

Información relacionada

[Expresiones](#)

Función UCASE (cadena)

Convierte en mayúsculas todos los caracteres de la cadena especificada.

Sintaxis

UCASE(<*string*>)

Descripción

Convierte en mayúsculas todos los caracteres de la cadena <*cadena*>.

La función UCASE es idéntica a la función UPPER.

Ejemplo

El siguiente ejemplo convierte la cadena indicada en mayúsculas y devuelve el valor ANT:

```
SELECT UCASE ('Ant') "ucase" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función UMINUS (numérica)

Devuelve el valor negado del argumento numérico especificado.

Sintaxis

UMINUS(<*number*>)

Descripción

Devuelve el valor negado del argumento numérico especificado.

Ejemplo

El siguiente ejemplo devuelve el valor 765 para "uminus":

```
SELECT UMINUS(-765) "uminus" FROM DUMMY;
```

Información relacionada

[Tipos de datos numéricos](#)

Función UNICODE (cadena)

Devuelve un entero que contiene el punto de código Unicode del primer carácter de la cadena especificada.

Sintaxis

```
UNICODE(<character>)
```

Descripción

Devuelve un entero que contiene el punto de código Unicode del primer carácter de la cadena <carácter>, o NULL si el primer carácter no es una codificación válida.

Ejemplo

El siguiente ejemplo devuelve el punto de código Unicode para el carácter dado (35):

```
SELECT UNICODE ('#') "unicode" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función UPPER (cadena)

Convierte en mayúsculas todos los caracteres de una cadena.

Sintaxis

```
UPPER(<string>)
```

Descripción

Convierte en mayúsculas todos los caracteres de la cadena <cadena>.

La función UPPER es idéntica a la función UCASE.

Ejemplo

Este ejemplo convierte el string indicado Ant en mayúsculas y devuelve el valor ANT:

```
SELECT UPPER ( 'Ant' ) "uppercase" FROM DUMMY;
```

Información relacionada

[Tipos de datos de cadena de caracteres](#)

Función UTCTOLOCAL (fecha y hora)

Convierte el cronomarcador especificado entre UTC y la hora local.

Sintaxis

```
UTCTOLOCAL ( <timestamp> [, <timezone> [, <timezone_dataset>[, <source_schema> ] ] ]  
            [, <error_clause> ] )
```

Elementos de sintaxis

<timestamp>

Especifica un cronomarcador en UTC que se convertirá en hora local.

<timestamp> ::= <timestamp>

<timezone>

Especifica un parámetro de string que contiene el huso horario que define la hora local. Si el huso horario local no se especifica explícitamente, se utilizará el huso horario local del sistema SAP HANA.

<timezone> ::= <string_literal>

<timezone_dataset>

Especifica el conjunto de datos en el que buscar el huso horario indicado. Si se especifica, debe fijarse en `platform`, que busca en el conjunto de datos proporcionado por el sistema operativo.

<timezone_dataset> ::= { sap | platform }

- `sap` busca en el conjunto de datos en las tablas de definición de husos horarios. Para utilizar este valor, importe un conjunto de datos de huso horario (consulte la nota SAP [1791342](#) ). Si utiliza este valor sin importar un conjunto de datos, la función devuelve un error.
- `platform` busca en el conjunto de datos proporcionado por el sistema operativo. Este es el valor predeterminado.

<source_schema>

Especifica el nombre del esquema de datos de huso horario que se va a utilizar.

<error_clause>

Especifica el tratamiento de errores para la función. Si no se especifica, se utiliza el valor predeterminado `ERROR ON ERROR`.

```
<error_clause> ::=
{ ERROR ON ERROR
| NULL ON ERROR
| DEFAULT '<value>' ON ERROR }
```

Descripción

Convierte la hora UTC(GMT) *<time>* a la hora local en un huso horario.

Utilice horas UTC en lugar de cronomarcadores locales. El uso de horas locales o la conversión entre husos horarios locales puede requerir un tratamiento adicional en el código de aplicación.

Si se utiliza una columna como segundo argumento (*<huso horario>*) y la columna contiene una cadena vacía, no se devuelve ningún resultado y se genera un error.

Ejemplos

El siguiente ejemplo devuelve el equivalente de hora local, en EST, de la hora UTC especificada:

```
SELECT UTCTOLOCAL (TO_TIMESTAMP('2012-01-01 01:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'EST') "utctolocal"
```

El siguiente ejemplo devuelve el equivalente de hora local de la hora UTC especificada, utilizando el conjunto de datos de huso horario *sap*:

```
SELECT UTCTOLOCAL (TO_TIMESTAMP('2012-01-01 01:00:00', 'YYYY-MM-DD HH24:MI:SS'), 'EST', 'sap') "utcto"
```

El siguiente ejemplo devuelve el equivalente de hora local del cronomarcador UTC actual mediante el esquema de datos de huso horario predeterminado *TTZZ_SYSTEM*:

```
SELECT UTCTOLOCAL (CURRENT_TIMESTAMP, 'EST', 'sap', TTZZ_SYSTEM) "utcto" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor NULL ya que "INVALID" no es un huso horario válido:

```
SELECT UTCTOLOCAL('2025-09-12 10:30:00', 'INVALID', 'sap' NULL ON ERROR) FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 01-01-2025 00:00:00 ya que "NO VÁLIDO" no es un huso horario válido:

```
SELECT UTCTOLOCAL('2025-09-12 10:30:00', 'INVALID', 'sap' DEFAULT '2025-01-01 00:00:00' ON ERROR) FROM DUMMY;
```

Información relacionada

[Función LOCALTOUTC \(fecha y hora\).](#)

VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS Función (seguridad)

Muestra los detalles de cada restricción de conexión si se permite un inicio de sesión para una condición específica.

Sintaxis

```
VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS(
  [ usergroup_name=> ] '<usergroup_name>'
  [, [ client_IP_address=> ] '<client_IP_address>'
  [, [ connect_timestamp=> ] '<connect_timestamp>'
  [, [ application_name=> ] '<application_name>'
  [, [ authentication_method=> ] '<method_name>' ] ] ] )
```

Elementos de sintaxis

<usergroup_name>

Especifica el nombre del grupo que se va a verificar.

<client_ip_address>

Especifica la dirección IP del cliente que se va a verificar. Si no se especifica o es NULL, se utiliza la dirección IP real.

<connect_timestamp>

Especifica la hora de inicio de sesión que se debe verificar. Si no se especifica o es NULO, se utiliza la hora real.

i Nota

Este parámetro está reservado para uso futuro y se ignorará.

<application_name>

Especifica el nombre de la aplicación que se va a verificar. Si no se fija o es NULL, se utiliza el nombre de aplicación del inicio de sesión real.

<method_name>

Define el método de autenticación.

```
<method_name> ::=
  { password
  | ldap
  | saml
  | jwt
  | x509 }
```

Descripción

Valida todas las restricciones de conexión individuales, activas e inactivas, de un grupo de usuarios. Una restricción de conexión que está desactivada siempre devuelve TRUE en la columna IS_CONNECT_ALLOWED. Si una restricción de conexión no incluye un tipo de restricción de conexión en particular, el valor de columna correspondiente es NULO.

Al ejecutar la función, puede utilizar los parámetros nombrados que llaman la convención para especificar solo un subconjunto de parámetros.

Para verificar si un usuario podrá conectarse correctamente, utilice la función VALIDATE_USERGROUP_CONNECT_RESTRICTION.

Ejemplo

El siguiente ejemplo se basa en las siguientes afirmaciones:

```
CREATE USERGROUP MY_GROUP CONNECT RESTRICTION (R1 IP ('1.2.3.4') APPLICATION ('hdbsql'));
ALTER USERGROUP MY_GROUP ADD CONNECT RESTRICTION (R2 IP ('9.9.9.9', '8.8.8.8'));
ALTER USERGROUP MY_GROUP ENABLE CONNECT RESTRICTION R1;
```

Las siguientes dos afirmaciones son equivalentes. Ambos devuelven detalles de restricción de conexión en el nombre del grupo de usuarios MY_GROUP, donde la dirección IP del cliente es 1.2.3.4, el tiempo de conexión es el 17 de agosto de 2023 y el nombre de la aplicación es hdbsql.

```
SELECT * from SYS.VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS('MY_GROUP', '1.2.3.4', CURRENT_TIMESTAMP);
SELECT * from SYS.VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS(usergroup_name=>'MY_GROUP', client_ip_address=>'1.2.3.4',
connect_timestamp=>'2023-08-17', application_name=>'hdbsql');
```

Salida:

USERGROUP_NAME	USERGROUP_ID	RESTRICTION_NAME	IS_ENABLED	IS_CONNECT_ALLOWED
MY_GROUP	158328	R1	VERDADERO	VERDADERO
MY_GROUP	158328	R2	FALSO	VERDADERO

CLIENT_IP_ADDRESS	IS_CLIENT_IP_ADDRESS_ALLOWED	CONNECT_TIMESTAMP
1.2.3.4	VERDADERO	2023-08-17 19:16:13.751000000
1.2.3.4	FALSO	2023-08-17 19:16:13.751000000

IS_CONNECT_TIMESTAMP_ALLOWED	APPLICATION_NAME	IS_APPLICATION_NAME_ALLOWED
NULO	hdbsql	VERDADERO
NULO	hdbsql	NULO

Información relacionada

- [Sentencia CREATE USERGROUP \(control de acceso\)](#)
- [Declaración ALTER USERGROUP \(control de acceso\)](#)
- [VALIDATE_USERGROUP_CONNECT_RESTRICTION Función \(seguridad\)](#)

VALIDATE_USERGROUP_CONNECT_RESTRICTION Función (seguridad)

Muestra si un intento de conexión sería posible dadas las restricciones de conexión activas de un grupo de usuarios.

Sintaxis

```
VALIDATE_USERGROUP_CONNECT_RESTRICTION(
  [ usergroup_name=> ]'<usergroup_name>'
  [, [ client_ip_address=> ]'<client_ip_address>'
  [, [ connect_timestamp=> ]'<connect_timestamp>'
```

```
[, [ application_name=> ] '<application_name>'
[, [ authentication_method=> ] '<method_name>' ] ] ] )
```

Elementos de sintaxis

<usergroup_name>

Especifica el nombre del grupo que se va a verificar.

<client_ip_address>

Especifica la dirección IP del cliente que se va a verificar. Si no se especifica o es NULL, se utiliza la dirección IP real.

<connect_timestamp>

Especifica la hora de inicio de sesión que se debe verificar. Si no se especifica o es NULL, se utiliza la hora real.

i Nota

Este parámetro está reservado para uso futuro y se ignorará.

<application_name>

Especifica el nombre de la aplicación que se va a verificar. Si no se fija o es NULL, se utiliza el nombre de aplicación del inicio de sesión real.

<method_name>

Define el método de autenticación.

```
<method_name> ::=
{ password
| ldap
| saml
| jwt
| x509 }
```

Descripción

Esta función evalúa todas las restricciones de conexión para el grupo de usuarios <usergroup_name> para los valores de parámetro indicados o sus valores predeterminados (en caso de que no se especifiquen). Al ejecutar la función, puede utilizar los parámetros nombrados que llaman la convención para especificar solo un subconjunto de parámetros.

Para verificar si una restricción de conexión individual permitirá una conexión de usuario, utilice la función `VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS`.

Ejemplo

El siguiente ejemplo se basa en las siguientes afirmaciones:

```
CREATE USERGROUP MY_GROUP CONNECT RESTRICTION (R1 IP ('1.2.3.4') APPLICATION ('hdbsql'));
ALTER USERGROUP MY_GROUP ENABLE CONNECT RESTRICTION R1;
```

Las siguientes dos afirmaciones son equivalentes. Ambos devuelven detalles de restricción de conexión en el nombre del grupo de usuarios MY_GROUP, donde la dirección IP del cliente es 1.2.3.4, el tiempo de conexión es el 17 de agosto de 2023 y el nombre de la

aplicación es hdlsql.

```
SELECT * from SYS.VALIDATE_USERGROUP_CONNECT_RESTRICTION( 'MY_GROUP', '1.2.3.4', CURRENT_TIMESTAMP,
SELECT * from SYS.VALIDATE_USERGROUP_CONNECT_RESTRICTION(usergroup_name=>'MY_GROUP', client_IP_addi
connect_timestamp=>'2023-08-17', application_name=>'hdbsql');
```

Salida:

USERGROUP_NAME	USERGROUP_ID	IS_CONNECT_ALLOWED	CONNECT_TIMESTAMP	APPLICATION_NAME
MY_GROUP	158328	VERDADERO	2023-08-17 19:16:13.751000000	hdbsql

Información relacionada

- [Sentencia CREATE USERGROUP \(control de acceso\)](#)
- [Declaración ALTER USERGROUP \(control de acceso\)](#)
- [VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS Función \(seguridad\)](#)

Función VAR (agregar)

Devuelve la varianza de la expresión dada como el cuadrado de la desviación estándar. Esta función también se puede utilizar como función de ventana.

Sintaxis

Función de agregación:

```
VAR( [ ALL | DISTINCT ] <expression> )
```

Función de ventana:

```
VAR( <expression> ) <window_specification>
```

Elementos de sintaxis

<expression>

Especifica los datos de entrada para la función.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada

TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL(p,s)	DECIMAL	REAL	DOUBLE
DECIMAL(9,6)	DECIMAL(11,6)	DECIMAL(16,6)	DECIMAL(25,6)	DECIMAL(p',s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- p' = 18, si p ≤ 18
- p' = 28, si 18 < p ≤ 28
- p' = 38, si 28 < p ≤ 38

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplo

Las siguientes sentencias crean una tabla con datos, por ejemplo:

```
DROP TABLE "MyProducts";
CREATE COLUMN TABLE "MyProducts"(
  "Product_ID" NVARCHAR(10),
  "Product_Name" NVARCHAR(100),
  "Category" NVARCHAR(100),
  "Quantity" INTEGER,
  "Price" DECIMAL(10,2),
  PRIMARY KEY ("Product_ID") );

INSERT INTO "MyProducts" VALUES('P1','Shirts', 'Clothes', 32, 20.99);
INSERT INTO "MyProducts" VALUES('P2','Jackets', 'Clothes', 16, 99.49);
INSERT INTO "MyProducts" VALUES('P3','Trousers', 'Clothes', 30, 32.99);
INSERT INTO "MyProducts" VALUES('P4','Coats', 'Clothes', 5, 129.99);
INSERT INTO "MyProducts" VALUES('P5','Purse', 'Accessories', 3, 89.49);
```

El siguiente ejemplo devuelve 2128,67, la desviación de los valores de la columna Precio:

```
SELECT VAR("Price") FROM "MyProducts";
```

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Función VAR_POP \(agregar\)](#)
- [Función VAR_SAMP \(agregar\)](#)
- [Expresiones](#)

Función VAR_POP (agregar)

Devuelve la varianza de población de una expresión.

Sintaxis

```
VAR_POP(<expression>)
```

Descripción

Devuelve la varianza de población de la expresión como la suma de los cuadrados de la diferencia de <expresión> de la media de <expresión>, dividida por el número de filas restantes.

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada							
TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL(p,s)	DECIMAL	REAL	DOUBLE
DECIMAL(9,6)	DECIMAL(11,6)	DECIMAL(16,6)	DECIMAL(25,6)	DECIMAL(p',s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- p' = 18, si p ≤ 18
- p' = 28, si 18 < p ≤ 28
- p' = 38, si 28 < p ≤ 38

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplos

El siguiente ejemplo devuelve 0 como varianza de población para la expresión especificada:

```
CREATE ROW TABLE RTABLE (A INT);
INSERT INTO RTABLE VALUES (1);
SELECT VAR_POP(A) "VARPOP" FROM RTABLE;
```

El siguiente ejemplo devuelve 0,25 como varianza de población para la expresión especificada:

```
INSERT INTO RTABLE VALUES (2);
SELECT VAR_POP(A) "VARPOP" FROM RTABLE;
```

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Función VAR \(agregar\)](#)
- [Función VAR_SAMP \(agregar\)](#)
- [Expresiones](#)

Función VAR_SAMP (agregar)

Devuelve la varianza de muestra de una expresión.

Sintaxis

```
VAR_SAMP(<expression>)
```

Descripción

Devuelve la varianza de muestra de la expresión como la suma de los cuadrados de la diferencia de *<expresión>* de la media de *<expresión>*, dividida por el número de filas restantes menos 1. Esta función es similar a VAR, la única diferencia es que devuelve NULL cuando el número de filas es 1.

El tipo de datos resultante para esta función depende del tipo de datos de entrada, como se describe en la tabla siguiente:

Tipo de resultado basado en entrada							
TINYINT	SMALLINT	INTEGER	BIGINT	DECIMAL (p, s)	DECIMAL	REAL	DOUBLE
DECIMAL (9, 6)	DECIMAL (11, 6)	DECIMAL (16, 6)	DECIMAL (25, 6)	DECIMAL (p', s)	DECIMAL	REAL	DOUBLE

i Nota

Para DECIMAL(p, s), el tipo de resultado DECIMAL(p', s) cumple las siguientes reglas de precisión:

- p' = 18, si p ≤ 18
- p' = 28, si 18 < p ≤ 28
- p' = 38, si 28 < p ≤ 38

Para obtener más información, consulte *Expresiones* en la sección *Información relacionada*.

Ejemplos

El siguiente ejemplo devuelve NULL como varianza de muestra de la expresión especificada:

```
CREATE ROW TABLE RTABLE (A INT);
INSERT INTO RTABLE VALUES (1);
SELECT VAR_SAMP(A) "VARSAMP" FROM RTABLE;
```

El siguiente ejemplo devuelve 0,5 como varianza de muestra de la expresión especificada:

```
INSERT INTO RTABLE VALUES (2);
SELECT VAR_SAMP(A) "VARSAMP" FROM RTABLE;
```

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Funciones agregadas de ventana](#)
- [Agregar funciones](#)
- [Función VAR \(agregar\)](#)
- [Función VAR_POP \(agregar\)](#)

Función VECTOR_EMBEDDING (vector)

Crea un vector incrustando a partir de un texto.

Sintaxis

Variante 1:

```
VECTOR_EMBEDDING(<text>, <text-type>, <model-and-version>[, [<remote_source_schema_name>.<remote_source_name>]
```

donde [<remote_source_schema_name>.<remote_source_name> se especifica como identificador.

Variante 2:

```
VECTOR_EMBEDDING(<text>, <text-type>, <model-and-version>[, <remote_source_schema_name>, <remote_source_name>]
```

donde <remote_source_schema_name> y <remote_source_name> se especifican como literales de cadena.

Elementos de sintaxis

<text>

El texto para el que se crea una incrustación.

<text_type>

El tipo del texto. Debe ser DOCUMENT o QUERY.

<model_and_version>

El nombre y la versión del modelo que se utiliza para crear la incrustación. El modelo y la versión deben estar separados por un punto (.). Si el nombre del modelo o la versión contiene un punto, la cadena correspondiente debe estar entre comillas; por ejemplo, si el nombre del modelo es mymodel y la versión es 3.2, el parámetro model_and_version debe fijarse en mymodel."3.2" o "mymodel"."3.2". Sin embargo, si su nombre de modelo contiene el número de versión; por ejemplo, mymodel-4.6, debe especificarlo como "mymodel-4.6", (de lo contrario, se interpretaría como "mymodel-4"."6").

<remote_source_schema_name>

El esquema del servicio de incrustación de fuente remota que se utiliza. En la variante 2, si no se utiliza ningún esquema, el valor debe fijarse en NULO o en una cadena vacía. Tenga en cuenta que actualmente, los servicios de incrustación de fuente remota solo se pueden crear sin un esquema fuente remoto.

<remote_source_name>

El nombre del servicio de incrustación de fuente remota que se utiliza. El valor debe transferirse como se indica en la sentencia CREATE REMOTE SOURCE.

<options>

Especifica una lista de parámetros en formato JSON que controlan la ejecución de la función. El JSON de entrada sigue una estructura definida con dos posibles miembros raíz:

- execution_params: determina la ejecución de una consulta en el lado del cliente.
- provider_params: define propiedades adicionales de la solicitud enviada al servicio de incrustación.

Para obtener una lista de valores posibles, consulte [Posibles <opciones> Parámetros para NLP \(sin una fuente remota\)](#) y [Posibles <opciones> Parámetros para AI Core \(con una fuente remota\)](#).

Ejemplo

```
{
  "execution_params": {
    "timeout": 60000
  },
  "provider_params": {
    "max_token_num": 768
  }
}
```

Descripción

Crea una incrustación vectorial para el texto pasado utilizando el modelo identificado por el modelo pasado y la cadena de versión.

Esta función utiliza un servicio de incrustación del servicio interno de procesamiento del lenguaje natural (NLP) o un servicio de incrustación en el hub de IA generativa de SAP AI Core.

Cuando se utiliza el servicio NLP interno, esta función requiere que el servicio NLP esté activado. Para obtener más detalles, consulte [Funciones adicionales de la base de datos de SAP HANA](#).

Cuando se utiliza un servicio de incrustación de SAP AI Core, se debe crear una fuente remota correspondiente. Para obtener detalles sobre la creación de fuentes remotas, consulte [Crear una fuente remota de hub de IA generativa](#). El nombre de la fuente remota debe pasarse a la función como identificador.

Un usuario debe tener privilegios de ejecución para una fuente remota.

Al incrustar párrafos de documentos, el <text-type> debería ser "DOCUMENT". Al incrustar consultas, <text-type> debería ser "QUERY".

La dimensión del vector devuelto depende del modelo utilizado.

Los modelos pueden tener límites en el número de tokens en el texto. Los tokens que superen el límite de tokens se ignorarán.

→ Recordar

Si no se utiliza ninguna fuente remota, esta función requiere que el servicio NLP esté activado. Para obtener más detalles, consulte [Funciones adicionales de la base de datos de SAP HANA](#). Si se utiliza una fuente remota, debe configurarse como se describe en [Crear una fuente remota de hub de IA generativa](#). Un usuario debe tener privilegios de ejecución para la fuente remota.

Modelos disponibles sin fuente remota

Nombre de modelo y versión	Dimensión del vector	Límite de token predeterminado	Límite máximo de tokens	Idiomas admitidos	Descripción
SAP_GXY.20250407	768	512	1024	de, en, es, fr, it, ja, pt, zh	El modelo SAP_GXY se basa en el modelo base de Roberta. Tiene

Nombre de modelo y versión	Dimensión del vector	Límite de token predeterminado	Límite máximo de tokens	Idiomas admitidos	Descripción
					soporte enriquecido en varios idiomas y capacidades de texto breve.
SAP_NEB.20240715	768	256	1024	de, en, es, fr, pt	El modelo de incrustación de texto de SAP HANA es un modelo basado en la estructura del transformador. Puede codificar eficazmente información de lenguaje textual en vectores, y los vectores codificados se pueden utilizar para tareas tales como recuperación de información, clasificación de texto, agrupación en clústeres y búsquedas de similitud semántica.

Para enumerar todos los modelos disponibles, utilice [la vista de sistema NLP_VECTOR_EMBEDDING_MODELS](#).

Modelos disponibles en una fuente remota

El siguiente procedimiento integrado se puede utilizar para enumerar todos los modelos que están disponibles para una fuente remota determinada.

- GET_REMOTE_SOURCE_AI_MODELS(<remote_source_name>, ?)

Parámetros <options> posibles para NLP (sin una fuente remota)

execution_params

Nombre de parámetro	Tipo de parámetro	Valor por defecto	Descripción
tiempo de espera	ENTERO	30000	Es el tiempo de espera de las solicitudes HTTP enviadas al servicio de incrustación, medido en milisegundos. Un valor de 0 desactiva el tiempo de espera.

provider_params

Nombre de parámetro	Tipo de parámetro	Valor por defecto	Descripción
número_token_máx	ENTERO	Específico de modelo	El número máximo de tokens utilizados por el servicio de incrustación.
pca_dim_num	ENTERO	768	La dimensión del vector de salida se puede controlar usando una capa lineal entrenada. Los parámetros de esta capa se optimizan directamente mediante el algoritmo de análisis de componentes principales (PCA). Los valores posibles son 64, 128, 256, 384, 512, 768.

Parámetros <options> posibles para AI Core (con una fuente remota)

provider_params

Nombre de parámetro	Tipo de parámetro	Valor por defecto	Descripción
dimensiones	ENTERO	Específico de modelo	La dimensión del vector de salida.
normalizar	BOOLEANO	falso	Controla si el vector de salida está normalizado o no.

i Nota

Los parámetros mencionados anteriormente son específicos del modelo y pueden no ser compatibles con determinados modelos. Encontrará más detalles sobre los parámetros en Orchestration v2 de SAP Business Accelerator Hub.

Autorizaciones EJECUTAR subvención

Para conceder permiso a un usuario o rol para ejecutar la función con una fuente remota, utilice la siguiente sentencia:

```
GRANT EXECUTE ON REMOTE SOURCE [<remote_source_schema_name>.<remote_source_name> TO <user_name>
```

Ejemplo

La siguiente consulta devuelve una incrustación vectorial para el texto Hello world!:

```
SELECT VECTOR_EMBEDDING('Hello world!', 'DOCUMENT', 'SAP_NEb.20240715') FROM DUMMY
```

La siguiente consulta devuelve una incrustación vectorial para el texto ¡Hola mundo! mediante una fuente remota con el nombre MY_GENAI_HUB_REMOTE_SOURCE:

```
SELECT VECTOR_EMBEDDING('Hello world!', 'DOCUMENT', 'text-embedding-ada-002.2', MY_GENAI_HUB_REMOTE_SOURCE)
```

La siguiente consulta devuelve una incrustación vectorial para el texto ¡Hola mundo! con un tiempo de espera de ejecución fijado en 2000 milisegundos:

```
SELECT VECTOR_EMBEDDING('Hello world!', 'DOCUMENT', 'SAP_NEB.20240715', '{ "execution_params": { "t
```

La siguiente sentencia creará una tabla para la que se generan automáticamente incrustaciones al insertarlas:

```
CREATE TABLE MY_EMBEDDINGS(  
  TITLE NVARCHAR(100),  
  PARAGRAPH_ID INT,  
  PARAGRAPH NVARCHAR(5000),  
  EMBEDDING REAL_VECTOR GENERATED ALWAYS AS  
    VECTOR_EMBEDDING(PARAGRAPH, 'DOCUMENT', 'SAP_NEB.20240715'))
```

Una consulta típica podría tener el aspecto de la siguiente sentencia:

```
SELECT TOP 10 TITLE, PARAGRAPH  
FROM MY_EMBEDDINGS  
ORDER BY COSINE_SIMILARITY(  
  VECTOR_EMBEDDING('When was SAP founded?', 'QUERY', 'SAP_NEB.20240715'),  
  EMBEDDING) DESC
```

Información relacionada

[Vista de sistema NLP_VECTOR_EMBEDDING_MODELS](#)

https://api.sap.com/api/ORCHESTRATION_API_v2/resource/orchestration.v2.endpoints.create_embeddings 

Función WEEK (fecha y hora)

Devuelve un número de semana entre 1 y 54 para la fecha especificada.

Sintaxis

WEEK(<date>)

Descripción

Devuelve el número de la semana del año como un número entero. Las semanas empiezan el lunes y terminan el domingo. Los primeros días del año hasta el primer domingo son la semana 1; los siete días que empiezan con el día posterior al primer domingo son la semana 2, etc.

Tanto las funciones WEEK como ISOWEEK devuelven el número de semana para una fecha especificada, pero el formato del resultado es bastante diferente, y las dos funciones pueden gestionar la primera semana del nuevo año de forma diferente. Por ejemplo, cuando se suministra la fecha 01/01/2017, la función SEMANA considera que la fecha forma parte de la **primera semana de 2017** y devuelve 1, mientras que ISOWEEK considera que la fecha forma parte de la **última semana de 2016** y devuelve 2016-W52.

Ejemplo

El siguiente ejemplo devuelve el valor 23 para el número de semana de la fecha especificada:

```
SELECT WEEK (TO_DATE('2011-05-30', 'YYYY-MM-DD')) "week" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 53 para el número de semana de la fecha especificada:

```
SELECT WEEK ('2016-12-31') FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 1 para el número de semana de la fecha especificada:

```
SELECT WEEK ('2017-01-01') FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 2 para el número de semana de la fecha especificada:

```
SELECT WEEK ('2017-01-02') FROM DUMMY;
```

Información relacionada

[Función ISOWEEK \(fecha y hora\)](#)

[Tipos de datos de fecha y hora](#)

Función WEEKDAY (fecha y hora)

Devuelve el día de la semana para la fecha especificada.

Sintaxis

```
WEEKDAY (<date>)
```

Descripción

Devuelve una representación en enteros del día de la semana para la fecha <date>. El valor de retorno va de 0 a 6, que representa de lunes(0) a domingo(6).

Ejemplo

El siguiente ejemplo devuelve el valor 4 como día de la semana de la fecha especificada:

```
SELECT WEEKDAY (TO_DATE ('2010-12-31', 'YYYY-MM-DD')) "week day" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función WEIGHTED_AVG (ventana)

Calcula un promedio móvil ponderado utilizando ponderaciones que disminuyen aritméticamente.

Sintaxis

```
WEIGHTED_AVG(<expression>) <window_specification>
```

Elementos de sintaxis

<expression>

<expresión> indica una columna numérica que se utiliza en el cálculo de la media variable ponderada.

En un marco de ventana en el que <n> representa el número real de elementos (FILAS o GRUPOS) en el marco de ventana y <i> representa el número de elementos de la serie, los factores de ponderación se calculan con el siguiente algoritmo:

$$\text{weight}(<i>) = 2 * (<n> + 1 - <i>) / (<n> * (<n> + 1))$$

Los factores de ponderación se aplican en orden inverso de modo que en un marco de ventana con n filas, el primer factor de ponderación se multiplica por el valor de la fila n, el segundo factor de ponderación con el valor de la fila n-1, etc. Al utilizar WEIGHTED_AVG con la unidad GROUPS, los valores dentro de un grupo se promedian primero aritméticamente antes de multiplicarlos por el factor de ponderación correspondiente.

<window_specification>

Define una ventana en los datos sobre los que opera la función. Para <window_Specification>, véase [Funciones de ventana y Especificación de ventana](#).

Descripción

Calcula un promedio móvil ponderado utilizando ponderaciones que disminuyen aritméticamente.

Ejemplos

```
CREATE TABLE weather (station INT, ts DAYDATE, temperature FLOAT);
INSERT INTO weather VALUES(1, '2014-01-01', 0.0);
INSERT INTO weather VALUES(1, '2014-01-02', 3.0);
INSERT INTO weather VALUES(1, '2014-01-03', 4.5);
INSERT INTO weather VALUES(1, '2014-01-04', 6.0);
INSERT INTO weather VALUES(1, '2014-01-05', 6.3);
INSERT INTO weather VALUES(1, '2014-01-06', 5.9);
INSERT INTO weather VALUES(2, '2014-01-01', 1.0);
INSERT INTO weather VALUES(2, '2014-01-02', 3.4);
INSERT INTO weather VALUES(2, '2014-01-03', 5.0);
INSERT INTO weather VALUES(2, '2014-01-04', 6.7);
INSERT INTO weather VALUES(2, '2014-01-05', 4.6);
INSERT INTO weather VALUES(2, '2014-01-06', 6.9);

SELECT ts, temperature, WEIGHTED_AVG(temperature) OVER (ORDER BY ts ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) FROM weather ORDER BY ts
```

ts	TEMPERATURA	WEIGHTED_AVG(temperatura) OVER (ORDER BY ts ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) FROM weather ORDER BY ts
01.01.2014	0	0
01.01.2014	1	0.6666666666666666
02.01.2014	3	2.3333333333333335
02.01.2014	3.4	3.2666666666666666

ts	TEMPERATURA	WEIGHTED_AVG(temperatura) OVER (ORDER BY ts ROWS BETWEEN 1 PRECEDING AND CURRENT ROW) FROM weather ORDER BY ts
03.01.2014	4.5	4.13333333333333
03.01.2014	5	4.83333333333333
04.01.2014	6	5.66666666666666
04.01.2014	6.7	6.46666666666667
05.01.2014	6.3	6.43333333333333
05.01.2014	4.6	5.16666666666666
06.01.2014	5.9	5.46666666666667
06.01.2014	6.9	6.56666666666666

Información relacionada

[Expresiones](#)
[Funciones de ventana y la especificación de ventana](#)

Función WIDTH_BUCKET (varios)

Devuelve el número de intervalo que se ha asignado al resultado de una expresión especificada.

Sintaxis

```
WIDTH_BUCKET(<expression>, <min_value>, <max_value>, <num_buckets>)
```

Elementos de sintaxis

- <expression>**
Especifica el valor para el que se crea el histograma. <expresión> debe ser un valor numérico o de fecha/hora, o un valor que se pueda convertir implícitamente en un valor numérico o de fecha/hora. Si <expresión> se evalúa como NULL, <expresión> devuelve NULL.
- <min_value>**
Especifica el rango mínimo de punto final para <expresión>. <min_value> debe ser un valor numérico o de fecha y hora y no se puede evaluar como NULO.
- <max_value>**
Especifica el rango máximo de punto final para <expresión>. <max_value> debe ser un valor numérico o de fecha y hora y no se puede evaluar como NULO.
- <num_buckets>**
Especifica el número de intervalos para distribuir valores entre. <num_buckets> debe ser un entero positivo.

Descripción

El número de intervalo devuelto es un número entero entre 0 y <num_buckets> +1. Esta función siempre devuelve <num_buckets> más dos intervalos adicionales: bucket 0 y <num_buckets> +1. El intervalo 0 contiene el recuento de valores inferiores a

<min_value> y el intervalo <num_buckets> +1 contiene el recuento de valores superiores o iguales a <max_value>.

Ejemplo

El siguiente ejemplo devuelve los números de intervalo de myTable:

```
CREATE TABLE myTable (myValues INT);
INSERT INTO myTable VALUES(1);
INSERT INTO myTable VALUES(2);
INSERT INTO myTable VALUES(3);
INSERT INTO myTable VALUES(4);
INSERT INTO myTable VALUES(5);

SELECT myValues, WIDTH_BUCKET( myValues, 2, 5, 2 ) AS "BucketNo" FROM myTable;
```

miValores	Nº interv.
1	0
2	1
3	1
4	2
5	3

Información relacionada

[Expresiones](#)

Función WORKDAYS_BETWEEN (fecha y hora)

Calcula el número de días laborables entre una fecha de inicio especificada y una fecha de fin especificada.

Sintaxis

```
WORKDAYS_BETWEEN(
  <factory_calendar_id>,
  <start_date>,
  <end_date>
  [, <source_schema>
  [, <table_name_suffix>,
  [, <client> ] ] ] )
```

Elementos de sintaxis

<factory_calendar_id>

Especifica el ID del calendario de fábrica.

<factory_calendar_id> ::= <string_literal>

<start_date>

Especifica la fecha de inicio. Puede utilizar un tipo DAYDATE o una cadena de formato de fecha (por ejemplo, '20140101' o '2014-01-01') para este parámetro.

`<start_date> ::= <string_literal> | <DAYDATE>`

<end_date>

Especifica la fecha de fin. Puede utilizar un tipo DAYDATE o una cadena de formato de fecha (por ejemplo, '20140101' o '2014-01-01') para este parámetro.

`<end_date> ::= <string_literal> | <DAYDATE>`

<source_schema>

Especifica el esquema que contiene la tabla Calendario de fábrica o las tablas Calendario de fábrica y calendario de días festivos.

`<source_schema> ::= <string_literal>`

Este parámetro se puede omitir si el esquema es el mismo que el esquema actual.

<table_name_suffix>

Especifica el sufijo añadido al nombre de tabla del calendario de fábrica estándar o a los nombres de tabla del calendario de fábrica y festivos estándar si se utiliza un conjunto alternativo de tablas. Por ejemplo, con el sufijo "_XYZ", la función accede a las tablas alternativas TFACS_XYZ o FHC_C_FCAL_XYZ en lugar de las tablas estándar TFACS o FHC_C_FCAL. Si se omite o está vacío, se utilizan las tablas estándar.

`<table_name_suffix> ::= <string_literal>`

<client>

Especifica el mandante utilizado para la implementación del calendario de fábrica y festivos dependiente del mandante. Si se omite o está vacía, la función intenta recuperar el cliente del contexto (parámetro de usuario), que se puede fijar en el contexto mediante ALTER USER <user_name> SET PARAMETER CLIENT = <client>. El mandante se ignora si la implementación del calendario de fábrica independiente del mandante está activa.

`<client> ::= <string_literal>`

Tipo de devolución

ENTERO

Descripción

Existen dos implementaciones diferentes para el cálculo de días laborables en sistemas basados en ABAP como SAP S/4HANA:

- El calendario de fábrica que almacena sus datos en una única tabla TFACS.
- El calendario de fábrica y festivos que almacena sus datos en varias tablas (FHC*).

Por defecto, las funciones WORKDAYS_BETWEEN suponen que el calendario de fábrica está activo. Una vez que la tabla FHC_CONFIG (o FHC_CONFIG <table_suffix> donde se especifica <table_suffix>) esté disponible en el sistema y contenga una entrada con KEY_FIELD = 'MIGRATION_COMPLETED' y VALUE = 'X' para el cliente especificado, las funciones WORKDAYS_BETWEEN suponen que el calendario de fábrica y festivos más reciente está activo.

Las tablas correspondientes deben estar disponibles en la base de datos SAP HANA para utilizar la función WORKDAYS_BETWEEN. En SAP BW, SAP CRM y SAP ERP que se ejecutan en una base de datos SAP HANA, estas tablas se encuentran en el esquema ABAP SAP<SID>. Para otros sistemas de base de datos SAP HANA, estas tablas se pueden replicar desde un sistema SAP Business Suite.

Cuando el calendario de fábrica está activo, WORKDAYS_BETWEEN accede a la tabla TFACS (o TFACS<table_suffix> si se especifica <table_suffix>). Cuando el calendario de fábrica y festivos está activo, WORKDAYS_BETWEEN accede a las siguientes tablas, que deben estar disponibles en el sistema con <table_suffix> añadido a sus respectivos nombres si se especifica <table_suffix>:

- FHC_C_FCAL
- FHC_C_FCAL_EXC
- FHC_C_HCAL
- FHC_C_HCAL_ASSGN
- FHC_C_HOL
- FHC_C_HOL_EASTER
- FHC_C_HOL_FIXED
- FHC_C_HOL_FLOAT
- FHC_C_HOL_FLDATE
- FHC_C_HOL_WKDAY
- FHC_CONFIG

Los cálculos se realizan con respecto a un calendario de fábrica con el ID <factory_calendar_id>.

Si la <start_date> es anterior o igual a la <end_date>, se devuelve el número de días laborables del período que comienza en la <start_date> y finaliza en la <end_date>.

Si la <start_date> es posterior a la <end_date>, se devuelve un número negativo de días laborables en el período que comienza en la <end_date> y finaliza en la <start_date>. El número de días devuelto en ese caso siempre incluye el día de <end_date>, pero excluye el día de <start_date>.

Ejemplos

Los siguientes ejemplos utilizan exclusivamente la tabla Calendario de fábrica para el mes de enero de 2014. Para obtener ejemplos de cómo utilizar el calendario de fábrica y de días festivos, consulte [Calendario de fábrica](#).

Campo de bits de TFACS	Día del mes	Motivo por el que no funciona
0	1	Día festivo
1	2	
1	3	
0	4	Fin de semana
0	5	Fin de semana
0	6	Día festivo

Campo de bits de TFACS	Día del mes	Motivo por el que no funciona
1	7	
1	8	
1	9	
1	10	
0	11	Fin de semana
0	12	Fin de semana
1	13	
1	14	
1	15	
1	16	
1	17	
0	18	Fin de semana
0	19	Fin de semana
1	20	
1	21	
1	22	
1	23	
1	24	
0	25	Fin de semana
0	26	Fin de semana
1	27	
1	28	
1	29	
1	30	
1	31	

El siguiente ejemplo devuelve el valor 1 para los días laborables entre las dos fechas especificadas. **Resultado:** se ha calculado un solo día laborable a partir del día 9 y finalizando el, pero sin incluir, el 10.

```
SELECT WORKDAYS_BETWEEN( '01', '2014-01-09', '2014-01-10' , 'FCTEST') "workdays" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor -1 para los días laborables entre las dos fechas especificadas. **Resultado:** se ha calculado un solo día laborable el día 10, excluido el día 9.

```
SELECT WORKDAYS_BETWEEN('01', '2014-01-10', '2014-01-09', 'FCTEST') "workdays" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 1 para los días laborables entre las dos fechas especificadas. **Resultado:** el día laborable individual se ha calculado el día 17. Se excluyen el período de fin de semana y la fecha de fin.

```
SELECT WORKDAYS_BETWEEN('01', '2014-01-17', '2014-01-20', 'FCTEST') "workdays" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor -1 para los días laborables entre las dos fechas especificadas. **Resultado:** el día laborable individual se ha calculado el día 20. Se excluyen el período de fin de semana y el día de la fecha de inicio.

```
SELECT WORKDAYS_BETWEEN('01', '2014-01-20', '2014-01-17', 'FCTEST') "workdays" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 20 para los días laborables entre las dos fechas especificadas. **Resultado:** el sistema tiene en cuenta los fines de semana (4, 5, 11, 12, 18, 19, 25 y 26) y los días festivos (1 y 6) y los excluye del período laborable.

```
SELECT WORKDAYS_BETWEEN('01', '2014-01-01', '2014-01-31', 'FCTEST') "workdays" FROM DUMMY;
```

El siguiente ejemplo devuelve los valores 30, 28, 25 y 20.

```
SET SCHEMA "SAPABC";
CREATE ROW TABLE MY_DATES (FCID NVARCHAR(2), STARTDATE DAYDATE, ENDDATE DAYDATE);
INSERT INTO MY_DATES VALUES ('01', '2014-01-01', '2014-02-14');
INSERT INTO MY_DATES VALUES ('01', '2014-04-01', '2014-05-14');
INSERT INTO MY_DATES VALUES ('01', '2014-07-01', '2014-08-05');
INSERT INTO MY_DATES VALUES ('01', '2014-10-01', '2014-10-30');
SELECT WORKDAYS_BETWEEN(FCID, STARTDATE, ENDDATE) "production duration" FROM MY_DATES;
```

Información relacionada

[Tipos de datos](#)

[Tipos de datos de fecha y hora](#)

[Declaración ALTER USER \(control de acceso\)](#)

Función XMLEXTRACT (varios)

Devuelve un elemento XML que coincide con la consulta XPath especificada.

Sintaxis

```
XMLEXTRACT(<XML_document>, <XPath_query> [, <NamespaceDeclarations>])
```

Elementos de sintaxis

<XML_document>

Especifica un documento XML del tipo CLOB, NCLOB, NVARCHAR.

<XPath_query>

Especifica una expresión XPath del tipo NVARCHAR.

<NamespaceDeclarations>

Especifica una declaración de área de nombres del tipo NVARCHAR.

Descripción

Devuelve el elemento XML coincidente. El valor de retorno es del tipo NVARCHAR o CLOB/NCLOB en función del tipo indicado para *<XML_document>*.

Si un elemento XML está vacío (por ejemplo, *<name></name>*), se devuelve un resultado vacío. Si no se encuentra un elemento XML, la función devuelve un error.

Ejemplo

La siguiente sentencia devuelve el elemento *<name>* de la posición 2 (el ejemplo devuelve *<name>Jar</name>*):

```
SELECT XMLEXTRACT(
  '<doc>
    <item><id>1</id><name>Box</name></item>
    <item><id>2</id><name>Jar</name></item>
  </doc>',
  '/doc/item[2]/name'
) FROM DUMMY;
```

La siguiente sentencia devuelve *<ns1:name>Jar</ns1:name>*:

```
SELECT XMLEXTRACT(
  '<doc xmlns:ns1="http://namespace1.sap.com" xmlns:ns2="http://namespace2.sap.com">
    <ns1:item><ns1:id>1</ns1:id><ns1:name>Box</ns1:name></ns1:item>
    <ns1:item><ns1:id>2</ns1:id><ns1:name>Jar</ns1:name></ns1:item>
    <ns2:item><ns2:id>3</ns2:id><ns2:name>Table</ns2:name></ns2:item>
  </doc>',
  '/doc/ns1:item[2]/ns1:name',
  'xmlns:ns1="http://namespace1.sap.com" xmlns:ns2="http://namespace2.sap.com"'
) FROM DUMMY;
```

Información relacionada

[Función XMLEXTRACTVALUE \(varios\)](#)

Función XMLEXTRACTVALUE (varios)

Devuelve un valor XML que coincide con la consulta XPath especificada.

Sintaxis

```
XMLEXTRACTVALUE(<XML_document>, <XPath_query> [,<NamespaceDeclarations>])
```

Elementos de sintaxis

<XML_document>

Especifica un documento XML del tipo NVARCHAR o NCLOB (o CLOB que es el alias de NCLOB).

<XPath_query>

Especifica una expresión XPath del tipo NVARCHAR.

<NamespaceDeclarations>

Especifica una declaración de área de nombres del tipo NVARCHAR.

Descripción

Devuelve el contenido de texto del elemento XML coincidente. El valor de retorno es del tipo NVARCHAR o NCLOB en función del tipo indicado para **<XML_document>**.

Si un elemento XML está vacío (por ejemplo, **<name></name>**), se devuelve un resultado vacío. Si no se encuentra un elemento XML, la función devuelve un error.

Esta función solo se admite para elementos individuales de nodos XML subordinados. Por ejemplo, puede utilizar XMLEXTRACTVALUE para consultar los elementos **<name>** y **<age>** en el documento XML siguiente, ya que se trata de elementos individuales de nodos subordinados. Sin embargo, no puede consultar los elementos **<parent>** o **<child>** del documento, ya que ambos elementos contienen varios nodos subordinados.

```
<parent>
  <child>
    <name>Tom</name>
    <age>18</age>
  </child>
</parent>
```

Ejemplo

El siguiente ejemplo extrae el valor del elemento **<name>** de la posición 2 (y devuelve Jar):

```
SELECT XMLEXTRACTVALUE(
  '<doc>
    <item><id>1</id><name>Box</name></item>
    <item><id>2</id><name>Jar</name></item>
  </doc>',
  '/doc/item[2]/name'
) FROM DUMMY;
```

El siguiente ejemplo extrae el valor del elemento **<name>** en la posición 2 (y devuelve Jar):

```
SELECT XMLEXTRACTVALUE(
  '<doc xmlns:ns1="http://namespace1.sap.com" xmlns:ns2="http://namespace2.sap.com">
    <ns1:item><ns1:id>1</ns1:id><ns1:name>Box</ns1:name></ns1:item>
    <ns1:item><ns1:id>2</ns1:id><ns1:name>Jar</ns1:name></ns1:item>
    <ns2:item><ns2:id>3</ns2:id><ns2:name>Table</ns2:name></ns2:item>
  </doc>',
  '/doc/ns1:item[2]/ns1:name',
  'xmlns:ns1="http://namespace1.sap.com" xmlns:ns2="http://namespace2.sap.com" '
) FROM DUMMY;
```

Información relacionada

[Función XMLEXTRACT \(varios\)](#)

Función XMLTABLE (cadena)

Crea una tabla relacional a partir de una cadena XML.

Sintaxis

```
XMLTABLE( [ <XML_namespace_clause>, ]
  <row_pattern> PASSING <XML_argument> COLUMNS <column_definitions>
  <error_option>
)
```

Elementos de sintaxis

<XML_namespace_clause>

Define el área de nombres XML.

```
XMLNAMESPACE (<XML_namespace>[, ...]
  [DEFAULT <default_namespace>]
)
```

<Área_nombre_XML>

Especifica el área de nombres XML.

```
<XML_namespace> ::= <namespace_url> AS <namespace_alias>
```

Cuando se especifica la cláusula XMLNAMESPACE, el resultado solo contiene los registros que pertenecen a las áreas de nombres especificadas. Si se especifica la URL del área de nombres (DEFAULT <namespace_url>), todas las cadenas XPath deben hacer referencia a la URL del área de nombres.

<Área de nombres por defecto>

Especifica el área de nombres por defecto. <default_namespace>, <namespace_url> y <namespace_alias> son constantes de cadena.

<row_pattern>

Describe el formato de xmlNodes en el XML con una expresión XPath.

```
<row_pattern> ::= <str_const>
```

<XML_argument>

Especifica el XML que se va a convertir en una tabla, definida como una constante de cadena o una referencia de columna.

```
<XML_argument> ::=
  <str_const>
  | <column_ref>
```

<str_const>

Especifica un string XML que se debe convertir en una tabla.

<column_ref>

Especifica una columna que contiene el XML que se va a convertir en una tabla.

<column_definitions>

Define las columnas en la tabla de resultados.

```
<column_definitions> ::=
  <column_name> <column_type> [, <column_name> <column_type>, ...]
```

<nombre_columna>

Especifica el nombre de la columna.

```
<column_name> ::= <identifier>
```

<tipo_columna>

Especifica un tipo de datos para la columna.

```
<column_type> ::=
  FOR ORDINALITY
  | <regular_column_return_type> PATH <column_pattern> [DEFAULT <str_const>]
  | <formatted_column_return_type> FORMAT XML PATH <column_pattern> [DEFAULT <str_const>]
```

Si la columna es una columna de ordinalidad, el valor para la fila *l* es *l*. El tipo de retorno de la columna de ordinalidad es BIGINT.

Si la columna es una columna regular, el valor para la fila *l* se puede definir de la siguiente manera:

- Si el valor de resultado no está vacío, el resultado proviene de aplicar *<row_pattern>* o *<column_pattern>* a *<XML_argumento>*.
- Si el valor de resultado está vacío, el resultado proviene del valor predeterminado de *<column_Definition>*.
- Si no se especifica ningún valor predeterminado, se devuelve NULL.
- Si se especifica FORMAT XML, el resultado contiene las etiquetas XML y la estructura XML en lugar de devolver el valor.

<tipo_devolución_columna_regular>

Especifica el tipo de retorno de la columna.

```
<regular_column_return_type> ::=
  BIGINT
  | DATE
  | DECIMAL
  | DOUBLE
  | INT
  | NVARCHAR (<int_const>)
  | SECONDDATE
  | SMALLDECIMAL
  | TIME
  | TIMESTAMP
```

<column_pattern>

Especifica la expresión XPath que se utilizará para extraer el valor de una columna.

```
<column_pattern> ::= <str_const>
```

<tipo_columna_formateado>

Especifica la lista de posibles tipos de retorno para la columna formateada en la tabla resultante.

```
<formatted_column_return_type> ::= NVARCHAR (<int_const>)
```

<error_option>

Devuelve una excepción cuando se produce un error durante la creación de la tabla.


```
<error_option> ::= ERROR ON ERROR
```

Si no se fija ERROR EN ERROR, se devuelve un resultado vacío. Si se especifica ERROR ON ERROR, se devuelve un error en tiempo de ejecución en lugar de un resultado PREDETERMINADO o vacío.

Descripción

Utilice la función XMLTABLE como sustitución de la cláusula FROM en una consulta para extraer los datos de una cadena XML. La función XMLTABLE devuelve los datos como una tabla relacional.

ERROR ON ERROR devuelve explícitamente un error en un error en tiempo de ejecución, ya que el comportamiento predeterminado es devolver NULL en casos de un error en tiempo de ejecución.

FORMAT XML extrae un resultado en formato XML.

La tabla resultante tiene una fila para cada elemento en la cadena XML, que es el resultado de aplicar el <row_pattern> al <XML_argumento>. Las filas de la tabla resultante tienen una columna para cada <column_Definition>, con el <column_name> y el <return n_type> especificados.

No puede llamar la función XMLTABLE en la cláusula FROM de una sentencia UPDATE o DELETE.

Ejemplo

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE('/doc/item' PASSING
'<doc>
  <item><id>10</id><name>Box</name></item>
  <item><id>20</id><name>Jar</name></item>
</doc>'
COLUMNS
ID INT PATH 'id',
NAME NVARCHAR(20) PATH 'name'
) as XTABLE;
```

ID	NOMBRE
10	Caja
20	Tarro

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE('/doc/item' PASSING
'<doc>
  <item><id>10</id><name>Box</name></item>
  <item><id>20</id><name>Jar</name></item>
</doc>'
COLUMNS
RN FOR ORDINALITY,
NAME NVARCHAR(20) PATH 'name'
) as XTABLE;
```

RN	NOMBRE
1	Caja
2	Tarro

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE('/doc/item' PASSING
'doc'
  <item><id>10</id><name>Box</name></item>
  <item><id>20</id><name>Jar</name></item>
</doc>'
COLUMNS
ID NVARCHAR(20) FORMAT XML PATH 'id',
NAME NVARCHAR(20) FORMAT XML PATH 'name'
) AS XTABLE;
```

ID	NOMBRE
<id>10</id>	<name>Box</name>
<id>20</id>	<name>Jar</name>

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE('/doc/item' PASSING
'doc'
  <item><id subid="one">10</id><name>Box</name></item>
  <item><id subid="two">20</id><name>Jar</name></item>
</doc>'
COLUMNS
ID NVARCHAR(20) PATH 'id/@subid',
NAME NVARCHAR(20) PATH 'name'
) AS XTABLE;
```

ID	NOMBRE
uno	Caja
dos	Tarro

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE(
XMLNAMESPACE('http://www.sap.com/xmltable' AS 'sap'),
'/sap:doc/sap:item' PASSING
'sap:doc xmlns:sap="http://www.sap.com/xmltable">
  <sap:item><sap:id>1</sap:id><sap:name>Box</sap:name></sap:item>
  <sap:item><sap:id>2</sap:id><sap:name>Jar</sap:name></sap:item>
</sap:doc>'
COLUMNS
ID INT PATH 'sap:id',
NAME NVARCHAR(20) PATH 'sap:name'
) as XTABLE;
```

ID	NOMBRE
1	Caja
2	Tarro

El siguiente ejemplo devuelve la siguiente tabla:

```
CREATE ROW TABLE XMLDATA (XML NVARCHAR(5000));
INSERT INTO XMLDATA VALUES (
'<sap:doc xmlns:sap="http://www.sap.com/xmltable">
  <sap:item><sap:id>1</sap:id><sap:name>Box</sap:name></sap:item>
  <sap:item><sap:id>2</sap:id><sap:name>Jar</sap:name></sap:item>
</sap:doc>');
INSERT INTO XMLDATA VALUES (
'<labs:doc xmlns:labs="http://www.saplabskorea.com/xmltable">
  <labs:item><labs:id>1</labs:id><labs:name>Apple</labs:name></labs:item>
  <labs:item><labs:id>2</labs:id><labs:name>Coconut</labs:name></labs:item>
</labs:doc>');

SELECT * FROM
XMLTABLE(XMLNAMESPACE('http://www.saplabskorea.com/xmltable' AS 'myns'),
'/myns:doc/myns:item' PASSING
XMLDATA.XML
COLUMNS
ID INT PATH 'myns:id',
NAME NVARCHAR(20) PATH 'myns:name'
) AS XTABLE;
```

ID	NOMBRE
1	Apple
2	Coco

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE(XMLNAMESPACE('http://www.sap.com/xmltable' AS 'myns', 'http://www.namespace.com/name' as
'/myns:doc/myns:item' PASSING
'<sap:doc xmlns:sap= "http://www.sap.com/xmltable" xmlns:ns2= "http://www.namespace.com/name">
<sap:item><sap:id>1</sap:id><sap:name>Box</sap:name><ns2:count>3</ns2:count></sap:item>
<sap:item><sap:id>2</sap:id><sap:name>Jar</sap:name><ns2:count>4</ns2:count></sap:item>
</sap:doc>'
COLUMNS
ID INT PATH 'myns:id',
NAME NVARCHAR(20) PATH 'myns:name',
COUNT INT PATH 'myns2:count'
) AS XTABLE;
```

ID	NOMBRE	RECuento
1	Caja	3
2	Tarro	4

El siguiente ejemplo devuelve la siguiente tabla:

```
SELECT * FROM
XMLTABLE(
XMLNAMESPACE('http://www.namespace.com/name' AS 'myns2' DEFAULT 'http://www.sap.com/xmltable'),
'/doc/item' PASSING
'<sap:doc xmlns:sap="http://www.sap.com/xmltable" xmlns:ns="http://www.namespace.com/name">
```

```
<sap:item><sap:id>1</sap:id><ns:name>Box</ns:name></sap:item>
<sap:item><sap:id>2</sap:id><ns:name>Jar</ns:name></sap:item>
</sap:doc>'
COLUMNS
ID INT PATH 'id',
NAME NVARCHAR(20) PATH 'myns2:name'
) AS XTABLE;
```

ID	NOMBRE
1	Caja
2	Tarro

Función YEAR (fecha y hora)

Devuelve el número de año de una fecha especificada.

Sintaxis

```
YEAR(<date>)
```

Descripción

Devuelve el número del año de la fecha <date>.

Ejemplo

El siguiente ejemplo devuelve el valor 2011 para el año de la fecha especificada:

```
SELECT YEAR (TO_DATE ('2011-05-30', 'YYYY-MM-DD')) "year" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Función YEARS_BETWEEN (fecha y hora)

Calcula el número de años entre dos fechas especificadas.

Sintaxis

```
YEARS_BETWEEN(<date_1>, <date_2>)
```

Descripción

Calcula el número de años comprendido entre <date_1> y <date_2>. Devuelve NULL si <date_1> o <date_2> es NULL.

Ejemplos

El siguiente ejemplo devuelve el valor 2 para los años entre las dos fechas especificadas:

```
SELECT YEARS_BETWEEN(TO_DATE('2001-01-01'), TO_DATE('2003-03-14')) "years_between" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor -2 para los años entre las dos fechas especificadas

```
SELECT YEARS_BETWEEN(TO_DATE('2003-10-03'), TO_DATE('2001-01-14')) "years_between" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 1 para los años entre las dos fechas especificadas

```
SELECT YEARS_BETWEEN(TO_DATE('2001-10-15'), TO_DATE('2003-01-14')) "years_between" FROM DUMMY;
```

El siguiente ejemplo devuelve el valor 1 para los años entre las dos fechas especificadas

```
SELECT YEARS_BETWEEN(TO_DATE('2001-10-13'), TO_DATE('2003-01-14')) "years_between" FROM DUMMY;
```

Información relacionada

[Tipos de datos de fecha y hora](#)

Agregar funciones

Las funciones de agregación son funciones analíticas que calculan un valor agregado basado en un grupo de filas.

Funciones admitidas:

- [Función AUTO_CORR \(agregar\)](#)
- [Función AVG \(agregar\)](#)
- [Función CORR \(agregar\)](#)
- [Función CORR_SPEARMAN \(agregado\)](#)
- [Función COUNT \(agregar\)](#)
- [Función CROSS_CORR \(agregar\)](#)
- [Función DFT \(agregar\)](#)
- [Función FIRST_VALUE \(agregar\)](#)
- [Función LAST_VALUE \(agregar\)](#)
- [Función MAX \(agregar\)](#)
- [Función MEDIAN \(agregar\)](#)
- [Función MIN \(agregado\)](#)
- [Función NTH_VALUE \(agregar\)](#)
- [Función STDDEV \(agregado\)](#)
- [Función STDDEV_POP \(agregar\)](#)

- [Función STDDEV_SAMP \(agregar\)](#).
- [Función STRING_AGG \(agregar\)](#).
- [Función SUM \(agregar\)](#).
- [Función VAR \(agregar\)](#).
- [Función VAR_POP \(agregar\)](#).
- [Función VAR_SAMP \(agregar\)](#).

Información relacionada

[Funciones agregadas de ventana](#)

Funciones de matriz

Las funciones de matriz toman matrices como entrada.

Funciones admitidas:

- [Función CARDINALITY \(matriz\)](#).
- [Función MEMBER_AT \(matriz\)](#).
- [Función SUBARRAY \(matriz\)](#).
- [Función TRIM_ARRAY \(matriz\)](#).

Funciones de conversión de tipo de datos

Las funciones de conversión de tipo de datos convierten datos de un tipo de datos a otro.

Las funciones de conversión de tipo de datos se utilizan para convertir argumentos de función de un tipo de datos a otro, o para probar si una conversión es posible.

i Nota

Tanto en las conversiones de tipo numérico implícitas como explícitas, estas funciones siempre truncan los dígitos menos significativos hacia cero.

Funciones admitidas:

- [Función CAST \(conversión de tipo de datos\)](#).
- [Función TO_ARRAY \(conversión de tipo de datos\)](#).
- [Función TO_BIGINT \(conversión de tipo de datos\)](#).
- [Función TO_BINARY \(conversión de tipo de datos\)](#).
- [Función TO_BLOB \(conversión de tipo de datos\)](#).
- [Función TO_BOOLEAN \(conversión de tipo de datos\)](#).

- [Función TO_CLOB \(conversión de tipo de datos\)](#)
- [Función TO_DATE \(conversión de tipo de datos\)](#)
- [Función TO_DATS \(conversión de tipo de datos\)](#)
- [Función TO_DECIMAL \(conversión de tipo de datos\)](#)
- [Función TO_DOUBLE \(conversión de tipo de datos\)](#)
- [Función TO_HALF_VECTOR \(conversión de tipo de datos\)](#)
- [Función TO_INT \(conversión de tipo de datos\)](#)
- [Función TO_INTEGER \(conversión de tipo de datos\)](#)
- [TO_JSON_BOOLEAN \(conversión de tipo de datos\)](#)
- [Función TO_NCLOB \(conversión de tipo de datos\)](#)
- [Función TO_NVARCHAR \(conversión de tipo de datos\)](#)
- [Función TO_REAL \(conversión de tipo de datos\)](#)
- [Función TO_REAL_VECTOR \(conversión de tipo de datos\)](#)
- [Función TO_SECONDDATE \(conversión de tipo de datos\)](#)
- [Función TO_SMALLDECIMAL \(conversión de tipo de datos\)](#)
- [Función TO_SMALLINT \(conversión de tipo de datos\)](#)
- [Función TO_TIME \(conversión de tipo de datos\)](#)
- [Función TO_TIMESTAMP \(conversión de tipo de datos\)](#)
- [Función TO_TINYINT \(conversión de tipo de datos\)](#)
- [Función TO_VARBINARY \(conversión de tipo de datos\)](#)
- [Función TO_VARCHAR \(conversión de tipo de datos\)](#)

Funciones de fecha y hora

Las funciones de fecha y hora realizan operaciones en tipos de datos de fecha y hora o información de fecha u hora de devolución.

Funciones admitidas:

- [Función ADD_DAYS \(fecha y hora\)](#)
- [Función ADD_MONTHS \(fecha y hora\)](#)
- [Función ADD_MONTHS_LAST \(fecha y hora\)](#)
- [Función ADD_NANO100 \(fecha y hora\)](#)
- [Función ADD_SECONDS \(fecha y hora\)](#)
- [Función ADD_WORKDAYS \(fecha y hora\)](#)
- [Función ADD_YEARS \(fecha y hora\)](#)

- [Función CURRENT_DATE \(fecha y hora\)](#)
- [Función CURRENT_MVCC_SNAPSHOT_TIMESTAMP \(fecha y hora\)](#)
- [Función CURRENT_TIME \(fecha y hora\)](#)
- [Función CURRENT_TIMESTAMP \(fecha y hora\)](#)
- [Función CURRENT_UTCDATE \(fecha y hora\)](#)
- [Función CURRENT_UTCTIME \(fecha y hora\)](#)
- [Función CURRENT_UTCTIMESTAMP \(fecha y hora\)](#)
- [Función DAYNAME \(fecha y hora\)](#)
- [Función DAYOFMONTH \(fecha y hora\)](#)
- [Función DAYOFYEAR \(fecha y hora\)](#)
- [Función DAYS_BETWEEN \(fecha y hora\)](#)
- [Función EXTRACT \(fecha y hora\)](#)
- [Función HOUR \(fecha y hora\)](#)
- [Función ISOWEEK \(fecha y hora\)](#)
- [Función LAST_DAY \(fecha y hora\)](#)
- [Función LOCALTOUTC \(fecha y hora\)](#)
- [Función MINUTE \(fecha y hora\)](#)
- [Función MONTH \(fecha y hora\)](#)
- [Función MONTHNAME \(fecha y hora\)](#)
- [Función MONTHS_BETWEEN \(fecha y hora\)](#)
- [Función NANO100_BETWEEN \(fecha y hora\)](#)
- [Función NEXT_DAY \(fecha y hora\)](#)
- [Función NOW \(fecha y hora\)](#)
- [Función QUARTER \(fecha y hora\)](#)
- [Función SECOND \(fecha y hora\)](#)
- [Función SECONDS_BETWEEN \(fecha y hora\)](#)
- [Función UTCTOLOCAL \(fecha y hora\)](#)
- [Función WEEK \(fecha y hora\)](#)
- [Función WEEKDAY \(fecha y hora\)](#)
- [Función WORKDAYS_BETWEEN \(fecha y hora\)](#)
- [Función YEAR \(fecha y hora\)](#)
- [Función YEARS_BETWEEN \(fecha y hora\)](#)

Funciones de jerarquía

Las funciones de jerarquía le permiten trabajar con datos jerárquicos, como tablas con filas dispuestas en un árbol o gráfico dirigido.

SAP HANA admite tres tipos de funciones de jerarquía: **generador**, **navegación** y **escalar**.

A continuación se muestra una lista de las funciones de jerarquía admitidas, para las que puede encontrar documentación para en *SAP HANA Cloud, Guía del desarrollador de jerarquía de base de datos de SAP HANA* junto con documentación completa para la función de jerarquía de SAP HANA.

Funciones de generador

- Función de generador HIERARCHY
- Función de generador HIERARCHY_LEVELLED
- Función de generador HIERARCHY_SPANTREE
- Función de generador HIERARCHY_TEMPORAL

Funciones de navegación

- Función de navegación HIERARCHY_ANCESTORS
- Función de navegación HIERARCHY_ANCESTORS_AGGREGATE
- Función de navegación HIERARCHY_DESCENDANTS
- Función de navegación HIERARCHY_DESCENDANTS_AGGREGATE
- Función de navegación HIERARCHY_SIBLINGS

Funciones escalares

- Función escalar HIERARCHY_COMPOSITE_ID

Información relacionada

[Guía del desarrollador de SAP HANA Cloud, jerarquía de base de datos de SAP HANA](#)

[Función de generador HIERARCHY](#)

[Función de navegación HIERARCHY_ANCESTORS](#)

[Función de navegación HIERARCHY_ANCESTORS_AGGREGATE](#)

[Función escalar HIERARCHY_COMPOSITE_ID](#)

[Función de navegación HIERARCHY_DESCENDANTS](#)

[Función de navegación HIERARCHY_DESCENDANTS_AGGREGATE](#)

[Función de generador HIERARCHY_LEVELLED](#)

[Función de navegación HIERARCHY_SIBLINGS](#)

[Función de generador HIERARCHY_SPANTREE](#)

[Función de generador HIERARCHY_TEMPORAL](#)

Funciones JSON

Las funciones JSON son funciones que devuelven u operan en datos JSON.

Las funciones escalares que pueden tomar cadenas o números también pueden tomar tipos de datos JSON como argumentos. Sin embargo, la evaluación de la función JSON verifica estrictamente el tipo de datos subyacente de JSON para proporcionar un comportamiento consistente. En otras palabras, no se permite la conversión implícita entre el tipo de datos inferior de JSON. Por ejemplo, `length(PARSE_JSON('123'))` genera una excepción porque la función `length()` debería tomar el tipo de datos string, donde `length(PARSE_JSON('123'))` es válido.

Funciones admitidas:

- [Función JSON_QUERY \(JSON\)](#)
- [Función JSON_TABLE \(JSON\)](#)
- [Función JSON_VALUE \(JSON\)](#)
- [Función PARSE_JSON \(JSON\)](#)
- [TO_JSON_BOOLEAN \(conversión de tipo de datos\)](#)

Otras funciones

SAP HANA admite muchas funciones que devuelven valores del sistema y realizan varias operaciones en valores, expresiones y valores de retorno de otras funciones.

Funciones admitidas:

- [Función ALLOW_PRECISION_LOSS \(varios\)](#)
- [Función COALESCE \(varios\)](#)
- [Función CONVERT_CURRENCY \(varios\)](#)
- [Función CONVERT_UNIT \(varios\)](#)
- [Función CURRENT_CONNECTION \(varios\)](#)
- [Función CURRENT_DATABASE \(varios\)](#)
- [Función CURRENT_IDENTITY_VALUE \(varios\)](#)
- [Función CURRENT_OBJECT_SCHEMA \(varios\)](#)
- [Función CURRENT_ROLEGROUP \(varios\)](#)
- [Función CURRENT_SCHEMA \(varios\)](#)
- [Función CURRENT_SITE_ID \(varios\)](#)
- [Función CURRENT_TRANSACTION_ISOLATION_LEVEL \(Otros\)](#)
- [Función CURRENT_USERGROUP \(varios\)](#)
- [Función CURRENT_UPDATE_STATEMENT_SEQUENCE \(varios\)](#)
- [Función CURRENT_UPDATE_TRANSACTION \(varios\)](#)
- [Función CURRENT_USER \(varios\)](#)
- [Función CURRENT_USER_ID \(varios\)](#)
- [Función EXPRESSION_MACRO \(varios\)](#)

- [Función GREATEST \(varios\)](#)
- [Función GROUPING \(varios\)](#)
- [Función GROUPING_ID \(varios\)](#)
- [Función HASH_MD5 \(varios\)](#)
- [Función HASH_SHA256 \(varios\)](#)
- [Función IFNULL \(varios\)](#)
- [Función LEAST \(varios\)](#)
- [Función MAP \(varios\)](#)
- [Función NEWUID \(varios\)](#)
- [Función NULLIF \(varios\)](#)
- [Función PARTITION_ALLOCATION \(varios\)](#)
- [PARTITION_HASH Función \(varios\)](#)
- [Función RECORD_COMMIT_TIMESTAMP \(varios\)](#)
- [Función RECORD_ID \(varios\)](#)
- [Función RESULT_CACHE_ID \(varios\)](#)
- [Función RESULT_CACHE_REFRESH_TIME \(varios\)](#)
- [Función SCORE \(varios\)](#)
- [Función SESSION_CONTEXT \(varios\)](#)
- [Función SESSION_USER \(varios\)](#)
- [Función STATEMENT_EXECUTION_HOST \(varios\)](#)
- [Función STATEMENT_EXECUTION_PORT \(varios\)](#)
- [Función SYSUID \(varios\)](#)
- [Función WIDTH_BUCKET \(varios\)](#)
- [Función XMLEXTRACT \(varios\)](#)
- [Función XMLEXTRACTVALUE \(varios\)](#)

Funciones numéricas

Las funciones numéricas realizan operaciones matemáticas en tipos de datos numéricos o información numérica de retorno.

Las funciones numéricas toman valores numéricos, o cadenas con caracteres numéricos, como entradas y valores numéricos de retorno. Cuando las cadenas con caracteres numéricos se proporcionan como entradas, la conversión implícita de una cadena a un número se realiza automáticamente antes de calcular los resultados.

Funciones admitidas:

- [Función ABS \(numérica\)](#)

- [Función ACOS \(numérica\)](#)
- [Función ASIN \(numérica\)](#)
- [Función ATAN \(numérica\)](#)
- [Función ATAN2 \(numérica\)](#)
- [Función BITAND \(numérica\)](#)
- [Función BITCOUNT \(numérica\)](#)
- [Función BITNOT \(numérica\)](#)
- [Función BITOR \(numérica\)](#)
- [Función BITSET \(numérica\)](#)
- [Función BITUNSET \(numérica\)](#)
- [Función BITXOR \(numérica\)](#)
- [Función CEIL \(numérica\)](#)
- [Función COS \(numérica\)](#)
- [Función COSH \(numérica\)](#)
- [Función COT \(numérica\)](#)
- [Función EXP \(numérica\)](#)
- [Función FLOOR \(numérica\)](#)
- [Función LN \(numérica\)](#)
- [Función LOG \(numérica\)](#)
- [Función MOD \(numérica\)](#)
- [Función NDIV0 \(numérica\)](#)
- [Función POWER \(numérica\)](#)
- [Función RAND \(numérica\)](#)
- [Función RAND_SECURE \(numérica\)](#)
- [Función ROUND \(numérica\)](#)
- [Función SIGN \(numérica\)](#)
- [Función SIN \(numérica\)](#)
- [Función SINH \(numérica\)](#)
- [Función SQRT \(numérica\)](#)
- [Función TAN \(numérica\)](#)
- [Función TANH \(numérica\)](#)
- [Función UMINUS \(numérica\)](#)

Funciones de seguridad

Las funciones de seguridad proporcionan una funcionalidad especial para fines de seguridad.

Funciones admitidas:

- [Función ESCAPE_DOUBLE_QUOTES \(seguridad\)](#)
- [Función ESCAPE_SINGLE_QUOTES \(seguridad\)](#)
- [Función GENERATE_PASSWORD \(seguridad\)](#)
- [IS_SQL_INJECTION_SAFE Función \(seguridad\)](#)
- [VALIDATE_USERGROUP_CONNECT_RESTRICTION_DETAILS Función \(seguridad\)](#)
- [VALIDATE_USERGROUP_CONNECT_RESTRICTION Función \(seguridad\)](#)

Funciones de datos de serie

Las funciones de datos de serie proporcionan una funcionalidad especial para los datos de serie y las tablas de serie.

Funciones admitidas:

- [Función SERIES_DISAGGREGATE \(datos de serie\)](#)
- [Función SERIES_ELEMENT_TO_PERIOD \(datos de serie\)](#)
- [Función SERIES_GENERATE \(datos de serie\)](#)
- [Función SERIES_PERIOD_TO_ELEMENT \(datos de serie\)](#)
- [Función SERIES_ROUND \(datos de serie\)](#)

Funciones de cadena

Las funciones de cadena realizan la extracción y manipulación de cadenas, o devuelven información sobre cadenas.

Funciones admitidas:

- [Función ABAP_ALPHANUM \(cadena\)](#)
- [Función ABAP_DF16RAW_TO_SMALLDECIMAL \(string\)](#)
- [Función ABAP_DF34RAW_TO_DECIMAL \(string\)](#)
- [Función ABAP_LOWER \(cadena\)](#)
- [Función ABAP_NUMC \(cadena\)](#)
- [Función ABAP_UPPER \(cadena\)](#)
- [Función ASCII \(cadena\)](#)
- [Función BINTOHEX \(cadena\)](#)

- [Función BINTONHEX \(cadena\)](#)
- [Función BINTOSTR \(cadena\)](#)
- [Función CHAR \(cadena\)](#)
- [Función CONCAT \(string\)](#)
- [Función CONCAT_NAZ \(cadena\)](#)
- [GET_DELTA_LAKE_TABLE_CHANGES Función \(string\)](#)
- [GET_DELTA_LAKE_TABLE_VERSION Función \(string\)](#)
- [Función HAMMING_DISTANCE \(string\)](#)
- [Función HEXTOBIN \(cadena\)](#)
- [Función HEXTONUM \(cadena\)](#)
- [Función INITCAP \(cadena\)](#)
- [Función LCASE \(cadena\)](#)
- [Función LEFT \(cadena\)](#)
- [Función LENGTH \(cadena\)](#)
- [Función LOCATE \(cadena\)](#)
- [Función LOCATE_REGEXPR \(string\)](#)
- [Función LOWER \(cadena\)](#)
- [Función LPAD \(cadena\)](#)
- [Función LTRIM \(cadena\)](#)
- [Función NCHAR \(cadena\)](#)
- [Función NORMALIZE \(cadena\)](#)
- [Función NUMTOHEX \(cadena\)](#)
- [Función OCCURRENCES_REGEXPR \(cadena\)](#)
- [Función PARSE_CERTIFICATES \(cadena\)](#)
- [Función REPLACE \(cadena\)](#)
- [Función REPLACE_REGEXPR \(cadena\)](#)
- [Función RIGHT \(cadena\)](#)
- [Función RPAD \(cadena\)](#)
- [Función RTRIM \(cadena\)](#)
- [Función SOUNDEX \(cadena\)](#)
- [Función STRTOBIN \(cadena\)](#)
- [SUBSTR_AFTER Función \(cadena\)](#)
- [Función SUBSTR_BEFORE \(cadena\)](#)
- [Función SUBSTRING \(cadena\)](#)

- [Función SUBSTRING_REGEXPR \(cadena\)](#)
- [Función TRIM \(cadena\)](#)
- [Función UCASE \(cadena\)](#)
- [Función UNICODE \(cadena\)](#)
- [Función UPPER \(cadena\)](#)
- [Función XMLTABLE \(cadena\)](#)

Funciones vectoriales

Las funciones vectoriales realizan cálculos en vectores.

Funciones admitidas:

- [Función CARDINALITY \(vector\)](#)
- [Función COSINE_SIMILARITY \(vector\)](#)
- [Función L2DISTANCE \(vector\)](#)
- [Función L2NORM \(vector\)](#)
- [Función L2NORMALIZE \(vector\)](#)
- [Función MEMBER_AT \(vector\)](#)
- [Función SUBVECTOR \(vector\)](#)
- [Función TO_ARRAY \(conversión de tipo de datos\)](#)
- [Función TO_BINARY \(conversión de tipo de datos\)](#)
- [Función TO_BLOB \(conversión de tipo de datos\)](#)
- [Función TO_HALF_VECTOR \(conversión de tipo de datos\)](#)
- [Función TO_NCLOB \(conversión de tipo de datos\)](#)
- [Función TO_NVARCHAR \(conversión de tipo de datos\)](#)
- [Función TO_REAL_VECTOR \(conversión de tipo de datos\)](#)
- [Función TO_VARBINARY \(conversión de tipo de datos\)](#)
- [Función VECTOR_EMBEDDING \(vector\)](#)

Información relacionada

[SAP HANA Cloud, guía del motor vectorial de la base de datos de SAP HANA](#)

Funciones de ventana y la especificación de ventana

Las funciones de ventana le permiten realizar operaciones analíticas en un conjunto de filas de entrada.

Sintaxis

<function_name> <window_specification>

<window_specification> ::= OVER ({ [<window_partition_by_clause>] [<window_order_by_clause>] |

Elementos de sintaxis

<function_name>

Especifica la función de ventana para operar sobre los datos. Consulte los temas de funciones individuales para obtener más información sobre su sintaxis.

```
<function_name> ::=
BINNING
| CUBIC_SPLINE_APPROX
| CUME_DIST
| DENSE_RANK
| LAG
| LEAD
| LINEAR_APPROX
| NTILE
| PERCENT_RANK
| PERCENTILE_CONT
| PERCENTILE_DISC
| RANDOM_PARTITION
| RANK
| ROW_NUMBER
| SERIES_FILTER
| WEIGHTED_AVG
| <window_aggregate_functions>
| <spatial_functions>
```

Para <window_attachate_functions>, véase [Ventana Funciones agregadas](#).

Para <spacal_functions>, consulte la guía *Referencia espacial de SAP HANA Cloud, base de datos de SAP HANA*.

<window_partition_by_clause>

<window_partition_by_clause> ::= PARTITION BY <expression> [{ , <expression> } ...]

Crea una o más particiones de ventana utilizando las expresiones proporcionadas.

<window_order_by_clause>

Define el orden de clasificación de la ventana.

<window_order_by_clause> ::= ORDER BY <window_order_by_expression> [{, <window_order_by_expression>

<window_order_by_expression> ::=
<column> [<collate_clause>] [ASC | DESC] [NULLS { FIRST | LAST }]

<collate_clause> ::= COLLATE <collation_name>

<clave_colar>

Especifica la intercalación que se va a utilizar. <collate_cláusula> solo se puede utilizar en columnas definidas como NVARCHAR (o VARCHAR, que es un alias para NVARCHAR).

<collation_name> es uno de los nombres de intercalación admitidos enumerados en la vista del sistema COLLATIONS.

[ASC | DESC]

ASC clasifica los registros en orden ascendente y DESC clasifica los registros en orden descendente. El valor predeterminado es ASC.

[NULLS FIRST | NULLS LAST]

Especifica el orden de los valores NULL.

<posición>

<position> utiliza las entradas de la lista de selección para definir el orden necesario. Por ejemplo:

```
SELECT col1, col2 FROM t ORDER BY 2;
```

ORDER BY 2 indica que el orden debe realizarse utilizando la segunda expresión en la lista de selección, que en este caso es col2.

<window_frame_clause>

Define las filas (límites) de una partición de ventana especificando el número de filas anteriores o siguientes con respecto a la fila actual. Si especifica una *<window_frame_cláusula>*, también debe especificar una *<window_order_by_cláusula>*.

```
<window_frame_clause> ::= <window_frame_unit> <window_frame_extent>
```

```
<window_frame_extent> ::= { <window_frame_start> | <window_frame_between> }
```

```
<window_frame_unit> ::= ROWS
```

```
<window_frame_start> ::= { UNBOUNDED PRECEDING | <window_frame_preceding> | CURRENT ROW }
```

```
<window_frame_preceding> ::= <unsigned_integer> PRECEDING
```

```
<window_frame_between> ::= BETWEEN <lower_window_frame_bound> AND <upper_window_frame_bound>
```

```
<lower_window_frame_bound> ::= <window_frame_bound>
```

```
<upper_window_frame_bound> ::= <window_frame_bound>
```

```
<window_frame_bound> := { <window_frame_start> | UNBOUNDED FOLLOWING | <window_frame_following> }
```

```
<window_frame_following> ::= <unsigned_integer> FOLLOWING
```

<window_frame_between> especifica el límite inferior (inicial) y el límite superior (final) del marco de ventana. El límite superior no puede ser menor que el límite inferior.

UNBOUND PRECEDING especifica que la ventana comienza en la primera fila de la partición. Solo se puede especificar como *<window_frame_start>*. FILA ACTUAL especifica que la ventana empieza o termina en la fila actual cuando se utiliza con FILAS.

PRECEDING indica que el número de filas o valores que deben preceder a la fila actual. *<unsigned_integer>* debe ser 0 o un literal de entero positivo.

UNBOUNDED FOLLOWING especifica que la ventana termina en la última fila de la partición. UNBOUNDED FOLLOWING solo se puede especificar como un fin de marco de ventana.

SIGUIENTE indica el número de filas o valores a seguir a la fila actual. Cuando se especifica como el inicio del marco de ventana, el fin del marco de ventana debe ser *<unsigned_integer>* SIGUIENTE. El *<unsigned_integer>* debe ser 0 o un literal de entero positivo.

La *<cláusula de marco de ventana>* es compatible con las funciones de ventana que realizan la agregación o la evaluación estadística en un rango de filas. Estas funciones utilizan el marco de ventana para definir qué filas incluir en el cálculo: AVG, COUNT, SUM, MIN, MAX, MEDIAN, FIRST_VALUE, LAST_VALUE, NTH_VALUE, STDDEV, STDDEV_POP, STDDEV_SAMP, VAR, VAR_POP, VAR_SAMP, CORR, CORR_SPEARD_AVG, WEAR_POP.

Las funciones que evalúan los valores en función de la posición de la fila, el desplazamiento relativo, la clasificación o la distribución utilizan el orden de las ventanas para determinar los resultados y no admiten el uso de una *<cláusula de marco de ventana>*. Estos incluyen: LEAD, LAG, ROW_NUMBER, RANK, DENSE_RANK, NTILE, CUME_DIST, PERCENT_RANK, PERCENTILE_CONT, PERCENTILE_DISC, CUBIC_SPLINE_APPROX, LINEAR_APPROX, funciones de ventana espacial (ST_) y funciones de ventana específicas de IBP (IBP_).

Descripción

Las funciones de ventana le permiten dividir los conjuntos de resultados de una consulta, o una partición lógica de una consulta, en grupos de filas llamadas particiones de ventana.

Una partición de ventana se especifica mediante una o más expresiones en la cláusula OVER.

Las funciones de ventana que no están en la siguiente lista deben tener una cláusula ORDER BY en la cláusula OVER:

- PROMEDIO
- BINNING
- CORR
- CORR_SPEARMAN
- RECUENTO
- MÁX.
- MEDIANO
- MÍN.
- PERCENTILE_CONT
- PERCENTILE_DISC
- ROW_NUMBER
- STDDEV
- SUMA
- VAR

Los conjuntos de resultados se particionan primero como se especifica en la cláusula PARTITION BY y, a continuación, se clasifican por la especificación de la cláusula ORDER BY dentro de la partición de ventana.

Finalmente, las funciones de ventana se aplican a cada fila dentro de los límites de partición de ventana.

La cláusula ORDER BY de la cláusula OVER solo se utiliza para evaluar la función de ventana de modo que el orden de las filas resultantes no sea determinista si no se especifica por ORDER BY para SELECT.

El marco de ventana predeterminado de la función de ventana depende de si se especifica o no una cláusula ORDER BY de ventana. Si se especifica una cláusula de ventana ORDER BY, el marco de ventana predeterminado se convierte en 'entre UNBOUNDED PRECEDING y CURRENT ROW'. Es decir, que la función de ventana calcula en filas anteriores o iguales a la fila actual. Como resultado, la función devuelve valores acumulados.

Si no se especifica una cláusula de ventana ORDER BY, el marco de ventana predeterminado se convierte en 'entre UNBOUNDED PRECEDING y UNBOUNDED FOLLOWING'. Es decir, que la función de ventana calcula en toda la partición de ventana. Como resultado, la función devuelve el mismo valor dentro de una partición de ventana independientemente de la fila actual.

Ejemplos

Los ejemplos utilizan la tabla de ejemplo T definida a continuación.

```
CREATE TABLE myTable (Class NVARCHAR(10), Value INT, Offset INT);
INSERT INTO myTable VALUES('A', 1, 1);
INSERT INTO myTable VALUES('A', 3, 3);
INSERT INTO myTable VALUES('A', 5, NULL);
```

```
INSERT INTO myTable VALUES('A', 5, 2);
INSERT INTO myTable VALUES('A', 10, 0);
INSERT INTO myTable VALUES('B', 1, 3);
INSERT INTO myTable VALUES('B', 1, 1);
INSERT INTO myTable VALUES('B', 7, 1);
```

```
SELECT Class, VALUE,
       ROW_NUMBER() OVER (PARTITION BY Class ORDER BY Value) AS ROW_NUM,
       RANK() OVER (PARTITION BY Class ORDER BY Value) AS RANK,
       DENSE_RANK() OVER (PARTITION BY Class ORDER BY Value) AS DENSE_RANK
FROM myTable;
```

clase	val	número_fila	clasificación	dense_rank
A	1	1	1	1
A	3	2	2	2
A	5	3	3	3
A	5	4	3	3
A	10	5	5	4
B	1	1	1	1
B	1	2	1	1
B	7	3	3	2

Información relacionada

- [Funciones agregadas de ventana](#)
- [Vista del sistema COLLATIONS](#)
- [SAP HANA Cloud, referencia espacial de base de datos de SAP HANA](#)
- [Funciones de Windows](#)

Funciones de Windows

Las funciones de ventana le permiten realizar operaciones analíticas en un conjunto de filas de entrada.

Funciones admitidas:

- [Función BINNING \(ventana\)](#)
- [Función CUBIC SPLINE APPROX \(ventana\)](#)
- [Función CUME_DIST \(ventana\)](#)
- [Función DENSE_RANK \(ventana\)](#)
- [Función LAG \(ventana\)](#)
- [Función LEAD \(ventana\)](#)
- [Función LINEAR APPROX \(ventana\)](#)
- [Función NTILE \(ventana\)](#)

- [Función PERCENT_RANK \(ventana\)](#)
- [Función PERCENTILE_CONT \(ventana\)](#)
- [Función PERCENTILE_DISC \(ventana\)](#)
- [Función RANDOM_PARTITION \(ventana\)](#)
- [Función RANK \(ventana\)](#)
- [Función ROW_NUMBER \(ventana\)](#)
- [Función SERIES_FILTER \(ventana\)](#)
- [Función WEIGHTED_AVG \(ventana\)](#)

Funciones de agregación de ventana

Algunas funciones agregadas se pueden utilizar como funciones de ventana sobre una especificación de ventana.

Sintaxis

```
<aggregate_function_or_expression_as_window_function> ::=
  <aggregate_function_name> ( <arguments> ) <window_specification>
```

```
<aggregate_function_name> ::=
AVG
| CORR
| CORR_SPEARMAN
| COUNT
| FIRST_VALUE
| LAST_VALUE
| MAX
| MEDIAN
| MIN
| NTH_VALUE
| STDDEV
| SUM
| VAR
```

Elementos de sintaxis

<aggregate_function_name>

Especifica la función de agregación o la expresión de agregación que se va a utilizar como función de ventana.

<arguments>

Especifica la entrada a la función. Consulte el tema de función individual para la definición de *<argumentos>*.

<window_specification>

Especifica la definición de ventana. Consulte [Funciones de ventana y la Especificación de ventana](#).

Descripción

Las funciones de agregación de ventana son funciones de agregación que se pueden utilizar como funciones de ventana.

Ejemplos

```
CREATE TABLE T (class NVARCHAR(10), val INT, offset INT);
INSERT INTO T VALUES('A', 1, 1);
INSERT INTO T VALUES('A', 3, 3);
INSERT INTO T VALUES('A', 5, null);
INSERT INTO T VALUES('A', 5, 2);
INSERT INTO T VALUES('A', 10, 0);
INSERT INTO T VALUES('B', 1, 3);
INSERT INTO T VALUES('B', 1, 1);
INSERT INTO T VALUES('B', 7, 1);

SELECT class,
       val,
       offset,
       COUNT(*) OVER (PARTITION BY class) AS c1,
       COUNT(offset) OVER (PARTITION BY class) AS c2,
       COUNT(*) OVER (PARTITION BY class ORDER BY val) AS c3,
       COUNT(offset) OVER (PARTITION BY class ORDER BY val) AS c4,
       MAX(val) OVER (PARTITION BY class) AS m1,
       MAX(val) OVER (PARTITION BY class ORDER BY val) AS m2
FROM T;
```

CLASE	VAL	OFFSET	C1	C2	C3	C4	M1	M2
A	1	1	5	4	1	1	10	1
A	3	3	5	4	2	2	10	3
A	5	nulo	5	4	4	3	10	5
A	5	2	5	4	4	3	10	5
A	10	0	5	4	5	4	10	10
B	1	3	3	3	2	2	7	1
B	1	1	3	3	2	2	7	1
B	7	1	3	3	3	3	7	7

Información relacionada

- [Funciones de ventana y la especificación de ventana](#)
- [Función AVG \(agregar\)](#)
- [Función CORR \(agregar\)](#)
- [Función CORR_SPEARMAN \(agregado\)](#)
- [Función COUNT \(agregar\)](#)
- [Función FIRST_VALUE \(agregar\)](#)
- [Función LAST_VALUE \(agregar\)](#)
- [Función MAX \(agregar\)](#)
- [Función MEDIAN \(agregar\)](#)
- [Función MIN \(agregado\)](#)
- [Función NTH_VALUE \(agregar\)](#)
- [Función STDDEV \(agregado\)](#)
- [Función SUM \(agregar\)](#)
- [Función VAR \(agregar\)](#)